# Adaptive SQS Visibility Timeout Controller

## Complete Project Explanation and Understanding Guide

How it works, why it was built this way, and how to explain it in a viva, presentation or interview

| | |
|---|---|
| **Covers** | Repository commit 4730bfb (16 Oct 2026) with the 22 Oct 2026 clean-up; simulation results in results/full and results/smoke; the real Amazon SQS run in results/aws_replication. |
| **Contents** | Overview, concepts, architecture, execution flow, logic, source code, algorithms, technologies, worked examples, testing and results, novelty, limitations, 41 questions and answers, quick revision, glossary. |
| **Evidence rule** | Every number is measured from the code or its saved results, and the source is named. Expected or proposed behaviour is labelled as such. |
| **Date** | 22 October 2026 |

## How to use this guide

This guide teaches how the project works, from the problem it solves down to individual lines of code. It was written after reading every source file, test, configuration file, document and result file in the repository, and after running small traced examples with the project's own code. Every number in it comes from one of those sources, and each one says where it came from.

| If you have… | Read |
|---|---|
| 10 minutes before a viva | Chapter 14 (Quick Revision), then the two-minute explanation at its end. |
| An hour | Chapters 1, 2, 3 and 13 (Questions and Answers). |
| A few hours | Everything in order. Chapters 5–7 are the deepest and most important. |
| A code question | Chapter 6 (Source Code) and Appendix B (file index). |

## Which version this covers

| Item | Version used |
|---|---|
| Source code | Git commit **4730bfb** on branch main (16 Oct 2026) plus the clean-up of 22 Oct 2026 (updated test, README, docs, requirements; not yet committed). The clean-up changed no simulator or policy code. |
| Simulation results | `results/full/` (20 seeds, generated 16 Oct 2026 06:28, after the last simulator change in commit 39a0510) and `results/smoke/`. |
| Real AWS results | The real Amazon SQS replication run of 22 Oct 2026, now stored in `results/aws_replication/` (config `"mode": "aws"`). |
| Tests | 110 tests collected by pytest; all 110 pass. |

## Conventions

- **Measured** means a number produced by running the code and saved in a results file. **Expected** or **proposed** means a design intention or a suggestion, not a measurement. The guide always says which.
- File references look like `src/simulator.py` and functions like `_dispatch()`.
- Coloured boxes: **Definition** (green) explains a term; **Key idea** (yellow) is something to remember for a viva; **Note** (blue) adds context; **Caution** (red) marks a limitation, inconsistency or something not verified.
- Times are in seconds. In the simulator they are *simulated* seconds: nothing actually waits.

# Table of Contents

**1. Project Overview . . . . . 7**

1.1 The project in one paragraph . . . . . 7

1.2 What problem does it solve? . . . . . 7

1.3 Why is it needed? . . . . . 8

1.4 Main objectives . . . . . 8

1.5 What does the project produce? . . . . . 8

1.6 A real-world analogy: the restaurant ticket rail . . . . . 9

1.7 What makes it useful or different? . . . . . 9

1.8 The project at a glance . . . . . 10

**2. Core Concepts and Fundamentals . . . . . 11**

2.1 Message queues, producers and consumers . . . . . 11

2.2 Amazon SQS (Simple Queue Service) . . . . . 11

2.3 Visibility timeout . . . . . 12

2.4 Receipt handles and receive counts . . . . . 12

2.5 At-least-once delivery and the three kinds of duplicates . . . . . 12

2.6 Idempotency and the ledger . . . . . 13

2.7 Worker failures: crashes and hangs . . . . . 13

2.8 Heartbeats, leases and the visibility budget . . . . . 13

2.9 Dead-letter queue, maxReceiveCount and poison messages . . . . . 14

2.10 Percentiles: P90 and P99 . . . . . 14

2.11 Job classes and the rolling window . . . . . 14

2.12 Cold start . . . . . 15

2.13 Discrete-event simulation . . . . . 15

2.14 Randomness, seeds and common random numbers . . . . . 15

2.15 Probability distributions used for job durations . . . . . 16

2.16 The duplicate–recovery trade-off and the Pareto front . . . . . 16

2.17 Statistical testing in brief . . . . . 16

2.18 How the concepts connect . . . . . 16

**3. Complete Project Architecture . . . . . 17**

3.1 Walking through the diagram . . . . . 18

3.2 Components and responsibilities . . . . . 18

3.3 How components communicate . . . . . 19

3.4 Main data structures . . . . . 19

3.5 The life of one message . . . . . 20

3.6 Replication architecture (real SQS) . . . . . 20

3.7 Technologies at a glance . . . 21

**4. Complete Working and Execution Flow . . . 22**

4.1 What happens when the project starts . . . 22
4.2 Inputs the system receives . . . 22
4.3 The experiment loop, step by step . . . 22
4.4 Inside one simulation run . . . 24
4.5 After all runs: analysis and output . . . 25
4.6 Errors, unexpected inputs and exceptional cases . . . 25
4.7 What the user ultimately sees . . . 26
4.8 The replication run, step by step . . . 26

**5. Complete Logic Explanation . . . 27**

5.1 Receiving a message: _dispatch() . . . 27
5.2 Visibility expiry: _on_visibility_expire() . . . 28
5.3 Completion, commit and delete: _on_complete() and _delete() . . . 29
5.4 Crash: _on_failure() . . . 29
5.5 Hang and release: _on_hang(), _release_hung(), _on_hang_detected() . . . 30
5.6 Heartbeat and extension: _on_heartbeat() and _apply_extension() . . . 30
5.7 Dead-letter logic . . . 31
5.8 Correlated outages . . . 31
5.9 Policy decision logic . . . 31
5.10 Learning from outcomes . . . 33
5.11 Measuring a run: summarize_run() . . . 33
5.12 Judging results: when is a hypothesis "supported"? . . . 34
5.13 The real-SQS worker: ReplicationRun._process() . . . 34

**6. Source Code Explanation . . . 35**

6.1 Folder structure . . . 35
6.2 Data model and inputs . . . 35
6.3 Policies (src/policies/) . . . 37
6.4 The simulator (src/simulator.py) . . . 38
6.5 Metrics and statistics . . . 39
6.6 Experiment runner (src/experiment.py) . . . 41
6.7 Replication (src/replication.py, replication_report.py, aws_sqs_adapter.py) . . . 41
6.8 How the functions interact: a call trace . . . 42

**7. Algorithms and Mathematical Logic . . . 43**

7.1 What is implemented and what is only discussed . . . 43
7.2 Percentile with linear interpolation . . . 43

7.3 The timeout formula . . . 44
7.4 Partial-history cold start . . . 44
7.5 Conformal quantile . . . 44
7.6 Cost-aware quantile . . . 44
7.7 Page–Hinkley change detection . . . 45
7.8 Adaptive heartbeat formulas . . . 45
7.9 Workload and failure randomness . . . 46
7.10 Statistics formulas . . . 46
7.11 Closed-form results used to verify the simulator . . . 46
7.12 Computational complexity . . . 46

**8. Technologies, Tools and Dependencies . . . 48**

8.1 Installation and configuration . . . 49

**9. Data Flow and Worked Examples . . . 50**

9.1 Example 1: three jobs with a 3-second timeout . . . 50
9.2 Example 2: the same jobs with a fixed heartbeat . . . 51
9.3 Example 3: the adaptive controller learning a class . . . 51
9.4 Example 4: crash and hang recovery . . . 52
9.5 Example 5: detecting a workload shift (Page–Hinkley) . . . 52
9.6 Example 6: why the realistic baseline goes stale . . . 53

**10. Testing, Evaluation and Results . . . 54**

10.1 How the project is tested . . . 54
10.2 Important test cases by type . . . 54
10.3 Evaluation design . . . 55
10.4 Measured results: main scenarios . . . 55
10.5 Primary hypotheses: 18 of 18 supported . . . 57
10.6 Real Amazon SQS replication . . . 57
10.7 What the results demonstrate . . . 59
10.8 What the results do not prove . . . 59
10.9 Known issues: fixed and remaining . . . 59

**11. Novelty and Design Decisions . . . 61**

11.1 What is distinctive about this project . . . 61
11.2 What already exists (and should be cited in a write-up) . . . 61
11.3 Design decisions and the reasons for them . . . 62
11.4 Implemented versus future work . . . 62

**12. Limitations and Future Improvements . . . 63**

12.1 Current limitations . . . 63

12.2 Suggested improvements and how each would help . . . 63

**13. Important Questions and Answers . . . 65**

13.1 Basic concepts . . . 65
13.2 Architecture . . . 65
13.3 Algorithms and implementation . . . 66
13.4 Code decisions . . . 67
13.5 Testing and results . . . 67
13.6 Limitations and novelty . . . 68
13.7 Challenging questions . . . 68

**14. Quick Revision . . . 69**

14.1 One-page summary . . . 69
14.2 Complete working flow . . . 69
14.3 Concepts to remember . . . 70
14.4 Key formulas and metrics . . . 70
14.5 Component-to-responsibility reference . . . 70
14.6 Most important files and functions . . . 70
14.7 Novelty in two sentences . . . 71
14.8 The two-minute explanation . . . 71

**Glossary . . . 72**

**Appendix A — Command Reference . . . 74**

**Appendix B — Source File Index . . . 75**

CHAPTER 1

# Project Overview

## 1.1 The project in one paragraph

This project studies a single setting in Amazon's message-queue service, SQS, called the **visibility timeout**, and asks whether a program can choose it automatically and better than a person can. When a worker takes a job from the queue, SQS hides that job for the visibility timeout. If the timeout is too short, the job reappears while the first worker is still busy, and a second worker does the same work again (a **duplicate**). If it is too long, a job whose worker crashed stays hidden for a long time before anyone retries it (slow **recovery**). The project builds a **controller** that learns how long each kind of job usually takes and sets the timeout from that, a fast **simulator** of SQS to test it, an **experiment and statistics pipeline** that compares it fairly with fixed settings, and a small **harness** that repeats the comparison on a real SQS queue.

> **Key idea**
> Too short a timeout causes duplicate work. Too long a timeout makes crash recovery slow. The project is about finding a good place between these two costs automatically, for each kind of job, while the workload changes.

## 1.2 What problem does it solve?

Most systems pick one visibility timeout (SQS's default is 30 seconds) and use it for every message. Real queues usually carry different kinds of jobs: a thumbnail might take 1 second, a video 15 seconds. One number cannot suit both:

| Timeout choice | What goes wrong | Example from this project's measured results |
|---|---|---|
| Too short (3 s) | Long jobs reappear and are processed two or three times. | `fixed_short`: 44% of processing attempts were duplicates (mixed workload, 8% crashes). |
| Too long (30 s) | After a crash the job waits almost the full timeout before a retry. | `fixed_long`: mean recovery after a crash 27.5 s. |
| One average for everything | Shorter than every long job, so long jobs duplicate. | `fixed_overall_mean`: 30.6% duplicates. |

Source: results/full/aggregate_metrics.csv, scenario mixed_with_failures, 20-seed means.

The project's answer is a timeout that is (1) **per job class**, (2) **learned from recent history** so it follows changes, and, in its strongest form, (3) **renewed by heartbeats** with a learned upper limit, so a short lease can cover long jobs while still noticing crashes quickly.

## 1.3 Why is it needed?

- **Duplicate work costs money and can cause harm.** Each duplicate uses CPU time. If the job has side effects (charging a card, sending an email) and is not idempotent, a duplicate is a real bug, not only a waste.
- **Slow recovery delays users.** A job stuck behind a crashed worker for 30 seconds is 30 seconds late.
- **Workloads change.** A timeout tuned last month goes stale when jobs get slower, and nobody notices until duplicates appear. This project measured that: a pre-tuned per-class timeout had 20.6% duplicates after a sudden shift, against 5.9% for the adaptive controller (results/full, sudden_shift).
- **Dead-letter queues amplify bad timeouts.** If a healthy but slow job keeps timing out, SQS can count it as a failing message and move it to a dead-letter queue even though it was processed. The project measures this too.

## 1.4 Main objectives

The README lists five objectives. In plain words:

1. Build a simulator that reproduces how SQS hides, re-shows, deletes and extends messages, without waiting in real time.
2. Implement several timeout policies that are easy to understand: fixed, per class, adaptive, heartbeat.
3. Measure the trade-off between duplicates and recovery time under repeatable workloads and failures.
4. Keep results logically correct (each job's result applied once) while still counting the duplicate work.
5. Offer a small, cheap, optional check on real SQS that never stores credentials.

**Research question** (README): *Can a workload-aware adaptive visibility-timeout policy reduce duplicate message processing while maintaining fast recovery from worker failures compared with fixed SQS visibility-timeout policies?*

**Hypothesis** (README): in changing or mixed workloads, a rolling per-class P90 policy moves toward a better duplicate/recovery trade-off than global fixed policies; a well-tuned static per-class policy may match or beat it when nothing changes; cold start and sudden shifts may hurt it temporarily.

## 1.5 What does the project produce?

| Output | Where | What it contains |
|---|---|---|
| Attempt log | `attempt_metrics.csv` | One row per processing attempt: who, when, how long, timeout used, duplicate flags, outcome. |
| Run summary | `run_metrics.csv` | One row per (scenario, policy, seed, sweep setting) with about 35 metrics. |
| Aggregates | `aggregate_metrics.csv` | Mean, standard deviation, t and bootstrap 95% intervals per policy and scenario; Pareto flag. |
| Hypothesis tests | `primary_hypotheses.csv` | 18 declared tests with Holm-adjusted p-values and a supported flag. |
| Exploratory tests | `paired_comparisons.csv` | Every policy against two reference policies, paired by seed. |
| Figures and report | `plots/*.png`, `report.html` | 12 charts and a readable HTML summary. |
| Real-SQS comparison | `results/aws_replication/` | Real versus simulated metrics, attempt log, 7 figures, HTML report. |

## 1.6 A real-world analogy: the restaurant ticket rail

Picture a busy kitchen. Orders arrive as paper tickets on a rail (the **queue**). A cook (a **worker**) takes a ticket. To stop two cooks making the same dish, the ticket is hidden under a timer (the **visibility timeout**). When the cook finishes, the ticket is thrown away (**delete**).

| Kitchen | SQS / this project |
|---|---|
| Timer runs out while the cook is still cooking; the ticket reappears and a second cook makes the same dish. | Visibility timeout expires → redelivery → **duplicate processing**. |
| The cook faints. The ticket stays hidden until the timer runs out. | Worker **crash**; recovery waits for the timeout. |
| The cook calls out "still working!" every few minutes, resetting the timer. | **Heartbeat** with ChangeMessageVisibility. |
| The cook freezes but keeps shouting "still working!" | **Hang**: heartbeats continue while no progress is made; only a **budget** stops it. |
| The head chef sets each timer from how long that dish took recently. | The **adaptive per-class controller**. |
| The pass checks off each order once, even if two plates arrive. | The **idempotency ledger**: the result is applied once. |
| A ticket that failed five times goes in a special tray. | **Dead-letter queue** after maxReceiveCount receives. |

## 1.7 What makes it useful or different?

The README is explicit: *"AWS use is not the novelty."* The contribution is the controller, the event model and a careful trade-off analysis. Specifically:

- **Per-class, rolling, interpretable control.** The timeout is a percentile of recent successful durations plus a margin. Anyone can check why a timeout has a given value.
- **Heartbeat with a learned budget.** The strongest policy keeps a short 5-second lease (fast crash detection), renews it while the worker is alive, and stops renewing after a per-class budget learned from the 99th percentile, so a hung worker is reclaimed after about that class's worst normal duration instead of a fixed global cap.
- **Fair, strong baselines.** It compares against an *oracle* static per-class timeout that has seen the answers and a *realistic* one tuned on separate pilot data. Beating weak baselines would prove little.
- **Careful statistics.** Common random numbers, paired tests across 20 seeds, hypotheses written down before testing, and Holm correction for multiple tests.
- **Checked against theory and reality.** The simulator is tested against closed-form results and the Erlang C formula, and a harness replays the comparison on a real SQS queue.

> **Caution**
> The underlying idea of setting a timeout from recent measurements is not new. TCP has set retransmission timeouts this way for decades, and lease/heartbeat designs are standard in distributed systems. The repository does not contain a literature review comparing the controller with these. Chapter 11 discusses this honestly.

## 1.8 The project at a glance

| Fact | Value | Source |
|---|---|---|
| Language | Python (3.10+ recommended; files use future annotations) | README, src/*.py |
| Source size | About 2,500 lines in src/ (simulator 393, experiment 474, replication 429) | wc -l |
| Tests | 110, all passing | pytest run, 22 Oct 2026 |
| Policies compared in the full run | 31 named policies (including sweep variants) | results/full/run_metrics.csv |
| Scenarios in the full run | 28 (main, hang, sweep, realism and DLQ scenarios) | configs/full.json |
| Seeds | 20 evaluation seeds (11, 22, …, 220); tuning seeds 1001–1010 kept separate | configs/full.json, methodology.md |
| Runs in the full experiment | 7,060 simulations, 200 jobs each, 10 workers | results/full/run_metrics.csv |
| Declared hypotheses | 18 (H1–H18), all supported in the full run | results/full/primary_hypotheses.csv |
| Real SQS check | 1 run per policy, 40 jobs, 4 workers, 4 policies | results/aws_replication |

CHAPTER 2

# Core Concepts and Fundamentals

This chapter explains every idea the project relies on, in an order where each one builds on the previous one. For each concept: what it is, why the project needs it, how it works, an example, and how it connects to the rest.

## 2.1 Message queues, producers and consumers

> **Definition — Message queue**
> A buffer that holds units of work (messages) between the program that creates them (the **producer**) and the programs that process them (the **consumers** or **workers**).

**Why needed:** the whole project is about how a queue hands work to workers. **How it works:** the producer sends messages; workers repeatedly ask for one, process it, and confirm it is done. The two sides never talk directly, so either can be slow, crash or scale independently. **Example:** a web server puts "resize photo 42" on a queue and returns immediately; a worker resizes it later. **In the project:** each `Message` is a job with an id, a job type and a processing duration; workers are `worker-00`, `worker-01`, …

## 2.2 Amazon SQS (Simple Queue Service)

> **Definition — Amazon SQS**
> Amazon Web Services' managed message-queue service. A **Standard** queue offers very high throughput and **at-least-once** delivery: every message is delivered at least once, sometimes more.

The project models a Standard queue and uses five SQS API calls:

| API call | What it does | Used in the project for |
|---|---|---|
| `SendMessage` | Put a message on the queue. | Submitting each job. |
| `ReceiveMessage` | Take a visible message and hide it for the visibility timeout. Returns a receipt handle. | A worker starting a job. |
| `ChangeMessageVisibility` | Reset the hidden period of a message you hold (needs its receipt handle). | Heartbeats; setting the per-class lease in the real run. |
| `DeleteMessage` | Remove the message for good (needs a receipt handle). | Finishing a job. |
| `GetQueueAttributes` | Read queue statistics such as the approximate number of messages. | Safety check: the real run refuses a non-empty queue. |

## 2.3 Visibility timeout

**Definition — Visibility timeout**

After a message is received, the time during which SQS hides it from other receivers. If it is not deleted before the timeout ends, it becomes visible again and can be received again.

**Why it exists:** SQS cannot know if a worker died. Hiding the message for a while and then showing it again is how SQS guarantees that a job whose worker vanished is eventually retried. **How it works:** each receive sets a deadline = receive time + timeout. At the deadline, if the message is still there, it becomes visible. **Example:** timeout 3 s, job takes 7 s, received at 0.5 s. At 3.5 s it reappears and a second worker takes it, while the first is still working (Chapter 9, Example 1, shows exactly this, traced by the simulator).

**Key idea**

The visibility timeout is the project's only control knob. Every policy in the project is just a different rule for choosing this number (and, for heartbeat policies, for extending it).

## 2.4 Receipt handles and receive counts

**Definition — Receipt handle**

A token SQS returns with each receive. It identifies *that particular delivery*. Deleting or extending a message needs the handle. Each new receive produces a new handle.

**Definition — ApproximateReceiveCount**

How many times SQS has delivered the message. 1 for the first delivery, 2 for the first redelivery, and so on.

**In the simulator:** every receive creates a new `uuid4` receipt and stores it as the job's `current_receipt`. An old attempt whose handle is no longer current cannot extend the lease (SQS rejects it) and its expiry no longer re-shows the message. The receive count drives duplicate flags, failure draws and the dead-letter rule.

**Caution**

Deletion is simplified. In the simulator, the first successful attempt of a job deletes it even if a newer delivery exists. On real SQS a delete with an old handle may succeed without guaranteeing removal. The README and architecture document declare this simplification.

## 2.5 At-least-once delivery and the three kinds of duplicates

Because of the timeout mechanism, the same job can be delivered more than once. The project separates three different things that are often all called "duplicates":

| Name in code | Plain meaning | Exact rule |
|---|---|---|
| `duplicate_delivery` | The job was received again. | Any receive after the first (receive_count > 1). |
| `duplicate_processing` | Two workers were working on the same job at the same time. | An attempt starts while another attempt of the same job is still active. |
| `duplicate_committed_result` | A second successful finish that was rejected. | A successful attempt reaches the ledger after the job was already committed. |

**The headline metric is duplicate processing**: the share of attempts that overlapped another attempt of the same job. A redelivery after a crash is a duplicate *delivery* but not duplicate *processing*, because nobody else is working on the job. That distinction matters: a crash retry is good, an overlap is waste.

## 2.6 Idempotency and the ledger

> **Definition — Idempotent operation**
> An operation that has the same effect whether it is applied once or many times.

**Why needed:** with at-least-once delivery, correctness requires that a job's result is applied once. **How it works here:** `InMemoryIdempotencyLedger.commit_once(job_id)` keeps a set of committed job ids behind a lock. The first caller gets `True` and the job is committed; later callers get `False`. **Example:** in Example 1, job-1 finishes three times (7.5 s, 10.5 s, 13.5 s); only the first commits. **Connection:** the ledger keeps results correct, so the remaining cost of a duplicate is wasted compute, which is what the project measures.

> **Key idea**
> Idempotency protects the *result*, not the *cost*. Duplicates still burn worker time, so reducing them still matters.

## 2.7 Worker failures: crashes and hangs

| | Crash | Hang |
|---|---|---|
| What happens | The worker process dies part-way through. | The worker stops making progress but stays alive. |
| Heartbeats | Stop at the moment of the crash. | Continue (the heartbeat thread is still alive). |
| When the job is retried | When the current lease expires. | When the lease can no longer be extended (budget or extension limit reached), or a liveness probe kills it. |
| Code | `FailureDecision(fails=True)`, event `fail` | `FailureDecision(hangs=True)`, event `hang` |
| Rates used | none 0, low 2%, medium 8%, high 20% per attempt | same levels, drawn separately |

**Why hangs matter:** without hangs, an unlimited heartbeat would be perfect: it never duplicates, and crashes are still detected within one short lease. A hung worker that keeps heartbeating would hide its job forever. That is the reason heartbeat policies need a **visibility budget**, and why the project added hangs in Phase 3.

## 2.8 Heartbeats, leases and the visibility budget

> **Definition — Heartbeat**
> A periodic call by a busy worker to ChangeMessageVisibility, pushing the message's deadline forward so it does not reappear while work continues.

> **Definition — Lease**
> The length of each hidden period: the initial timeout and each extension.

> **Definition — Visibility budget**
> A cap on total hidden time measured from the start of processing. No extension may go past start + budget. It limits how long a hung worker can hold a job.

**Example** (`fixed_heartbeat`: lease 5 s, heartbeat every 3.5 s, at most 4 extensions, budget 40 s): the heartbeats at 3.5, 7, 10.5 and 14 s set deadlines of 8.5, 12, 15.5 and 19 s. So this policy covers jobs up to 19 s; a longer healthy job is redelivered at 19 s. A crash is noticed within one 5 s lease. **Cost:** each extension is one extra API call (about 4.1 versus 3.1 calls per completed job in the mixed scenario).

## 2.9 Dead-letter queue, maxReceiveCount and poison messages

> **Definition — Dead-letter queue (DLQ)**
> A separate queue where SQS moves a message after it has been received more than maxReceiveCount times, so a message that always fails stops being retried forever.

> **Definition — Poison message**
> A message that fails on every attempt (for example a malformed payload). Only a DLQ removes it.

**Why it matters here:** a too-short timeout makes a *healthy* slow job reappear repeatedly, so it can hit maxReceiveCount and be dead-lettered by mistake. The project counts these as `false_dlq_rate`. Measured: with maxReceiveCount = 2, `fixed_short` dead-lettered 23% of healthy jobs (README, Phase 5).

## 2.10 Percentiles: P90 and P99

> **Definition — Percentile**
> The value below which a given percentage of observations fall. P90 is the value that 90% of observations are at or below.

**Why P90:** it covers most jobs (so few overrun the timeout) without waiting for the very worst case (which would make crash recovery slow). The README calls it interpretable and says the percentile should be sensitivity-tested; the project sweeps P75, P80, P90, P95 and P99. **How computed:** NumPy's default linear interpolation between sorted values (Section 7.2). **Example:** durations 12.4, 13.1, 14.0, 15.8, 17.2 → P90 = 16.64 (computed by the code).

## 2.11 Job classes and the rolling window

> **Definition — Job class**
> The `job_type` field of a message: short, medium, long or heavy. The controller keeps separate statistics per class.

> **Definition — Rolling window**
> Only the most recent W successful durations of each class are kept (W = 40 in the main experiments, 30 in the smoke run). Older values drop out automatically.

**How:** a Python `deque(maxlen=W)` per class. Appending the 41st value silently removes the oldest. **Why:** a window lets the estimate follow changes. Small windows react fast but are noisy; large windows are stable but slow.

Measured: after a shift, windows of 10–25 beat 100 (README, Phase 2 sweep).

## 2.12 Cold start

> **Definition — Cold start**
> The period before the controller has enough history for a class (fewer than `min_history` = 5 samples) to compute a percentile.

The original rule used a fixed 12 s fallback, which is shorter than long jobs (median 14 s), so early long jobs duplicated. The current default ("partial" cold start) uses 1.5 × the largest sample seen so far + margin when 1–4 samples exist. Measured effect (H5): duplicates in the mixed, no-failure scenario fell by 0.95 percentage points, Holm p = 0.005.

## 2.13 Discrete-event simulation

> **Definition — Discrete-event simulation (DES)**
> A simulation that jumps from one event to the next in time order, instead of advancing a clock in small steps. Nothing happens between events, so they can be skipped instantly.

**Why:** 7,060 runs of 200 jobs each would take days in real time. With DES they take minutes. **How:** events are kept in a priority queue (a heap) ordered by (time, insertion counter). The loop pops the earliest event, sets `now` to its time, handles it, then hands visible messages to idle workers. The insertion counter breaks ties, so the order is deterministic. **Example:** a job received at 0.5 s with a 7 s duration and a 3 s timeout creates a `complete` event at 7.5 and a `visibility_expire` event at 3.5.

## 2.14 Randomness, seeds and common random numbers

> **Definition — Seed**
> A number that fixes the sequence produced by a random number generator. Same seed, same numbers.

> **Definition — Common random numbers (CRN)**
> Giving every policy the same random draws for the same situation, so that differences between policies come from the policies and not from luck.

**How the project does it:** the failure outcome of the k-th delivery of a job is drawn from a generator keyed by `(seed, crc32(job_id), k)`. So delivery 1 of job-00042 crashes (or not) identically under every policy. Workloads are generated once per (scenario, seed) and shared by all policies. Worker speeds, retry jitter and outages are also keyed by the seed. **Why it matters:** it makes *paired* comparisons possible, which are far more sensitive than comparing independent averages.

## 2.15 Probability distributions used for job durations

| Distribution | Shape | Used for | Parameters in the code |
|---|---|---|---|
| Log-normal | Positive, right-skewed, most values near the median. | short, medium, long jobs | median 1.2 / 5 / 14 s; log-spread σ 0.22 / 0.28 / 0.32 |
| Pareto (+1, scaled) | Heavy tail: rare, very long jobs. | heavy jobs | shape 2.2, scale 3 s → minimum 3 s |
| Exponential | Memoryless gaps between random events. | arrival gaps | rate 1.2 jobs per second (mean gap 0.83 s) |

All durations are clipped to [0.15 s, 90 s]. In the sudden-shift workload every job is "medium" and durations double (median 5 → 10 s) from the halfway point.

## 2.16 The duplicate–recovery trade-off and the Pareto front

> **Definition — Pareto optimal**
> A policy is Pareto optimal if no other policy is at least as good on both metrics and strictly better on one. The set of such policies is the Pareto front.

Lower duplicates and lower recovery time are both good, and usually you cannot improve one without hurting the other by changing a single fixed timeout. Plotting each policy as a point (recovery, duplicates) shows the trade-off; points on the front are the sensible choices. The code flags them in `pareto_optimal_dup_vs_recovery`.

## 2.17 Statistical testing in brief

- **Paired comparison:** for each seed, subtract policy B's metric from policy A's. Analyse the 20 differences.
- **Wilcoxon signed-rank test:** a test on those differences that does not assume they are normally distributed. It ranks their sizes and checks whether positive and negative ones balance.
- **p-value:** how surprising the observed differences would be if the two policies were really equal.
- **Multiple testing and Holm correction:** running many tests produces some false positives by luck. Holm's method raises p-values in a step-down way so the chance of any false positive in a family stays below 5%.
- **Pre-declared hypotheses:** writing down the exact test (scenario, metric, policies, direction) before running it, so the result cannot be cherry-picked. The project has 18 (H1–H18).
- **Confidence interval:** a range likely to contain the true mean; the project reports Student-t and bootstrap versions.

All of these are implemented by hand in `src/stats.py` and `src/metrics.py` (no SciPy). Chapter 7 explains each formula.

## 2.18 How the concepts connect

The workload generator creates jobs from the distributions. The simulator plays them through a model of SQS, where the visibility timeout (chosen by a policy) decides when duplicates and slow recoveries happen. The failure model injects crashes and hangs with common random numbers. The ledger keeps results correct while duplicate work is counted. Metrics turn the attempt log into duplicate rates, recovery times and more; statistics compare policies across seeds. Heartbeats, budgets, DLQs, cold start, change detection, conformal and cost-aware rules are refinements of the one decision: *how long should this message stay hidden?*

CHAPTER 3

# Complete Project Architecture

The project is a Python package, `src`, with two entry points: the **experiment runner** (`python -m src.experiment`), which runs simulations, and the **replication harness** (`python -m src.replication`), which runs the same policies on a real or fake SQS queue. Everything communicates by ordinary Python function calls and plain data objects; there is no server, database or network except the optional SQS calls.

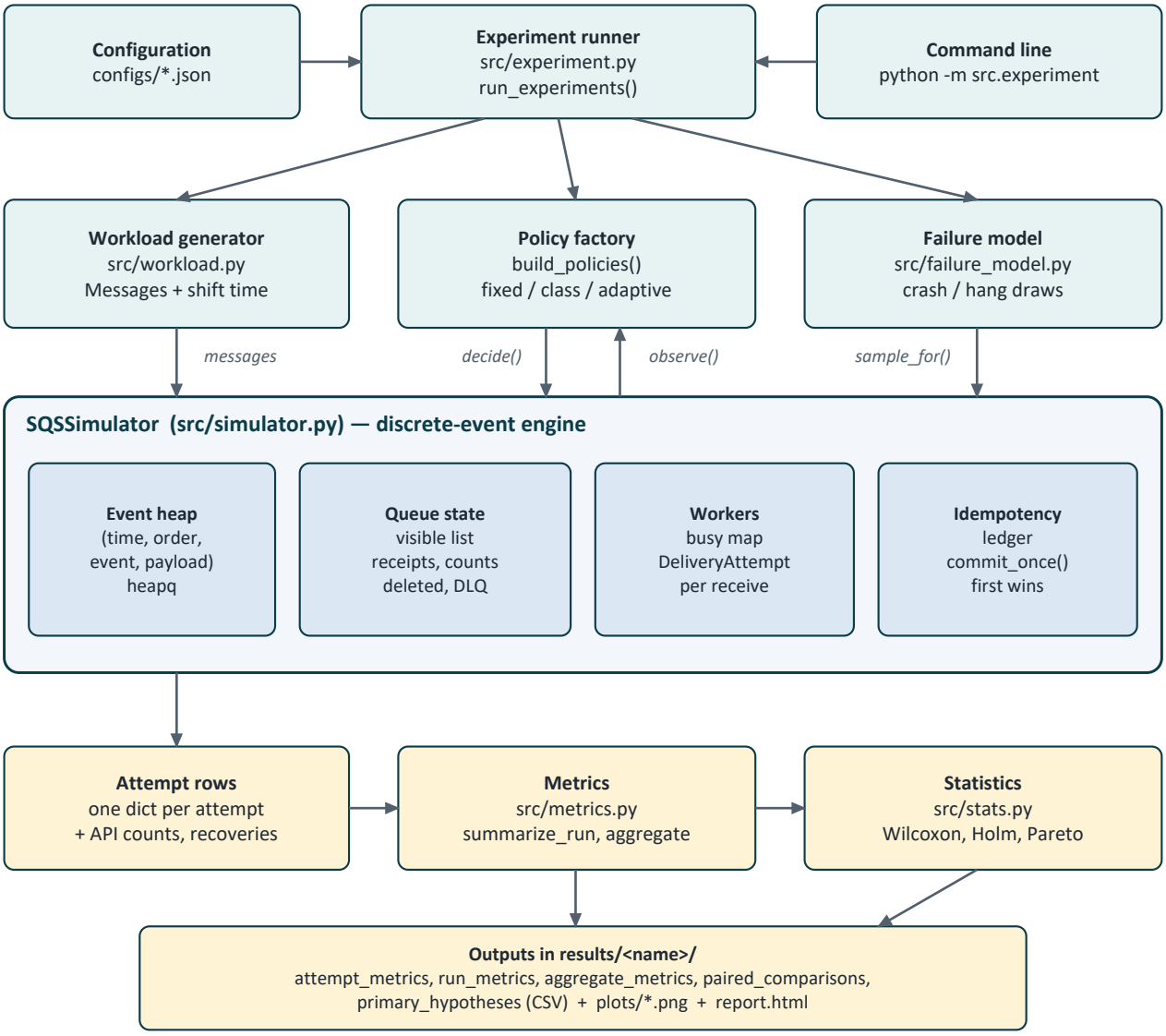



**Figure 1.** Simulation architecture. Green: inputs and builders. Blue: the simulator and its internal state. Yellow: analysis and outputs. Arrows show the direction data flows.

## 3.1 Walking through the diagram

1. **Configuration** (`configs/smoke.json`, `configs/full.json`) lists the number of jobs, workers, arrival rate, seeds, scenarios (workload + failure level + optional realism), sweeps and the 18 declared hypotheses. It is the only place experiments are described; policy code contains no experiment settings.
2. The **experiment runner** (`run_experiments()`) loops over scenarios and seeds. For each pair it asks the **workload generator** for a list of `Message` objects (and the shift time, for the shift workload).
3. The **policy factory** (`build_policies()`) creates fresh policy objects for that workload: fixed, per-class (oracle and pre-tuned), adaptive, heartbeat and all variants.
4. For each policy, a new **SQSSimulator** is created with the messages, the policy and a **FailureModel** for the scenario's crash and hang levels. Running it plays every job through the queue model.
5. Inside the simulator, the **event heap** orders what happens next; **queue state** tracks which messages are visible, deleted or dead-lettered; **workers** hold one `DeliveryAttempt` each; the **ledger** commits each job once. The simulator calls the policy (`decide()` on each receive; `observe()` after each committed success) and the failure model (`sample_for()` on each receive).
6. The simulator returns **attempt rows** plus counters. **Metrics** (`summarize_run()`) turn them into one row of about 35 numbers. After all runs, `aggregate_runs()` and the **statistics** functions produce means, intervals, paired tests and hypothesis results.
7. Everything is written to `results/<name>/` as CSV files, PNG plots and an HTML report.

## 3.2 Components and responsibilities

| Component | File | Responsibility | Input → Output |
|---|---|---|---|
| Data model | `src/message.py` | Define a job and one delivery attempt. | — → Message, DeliveryAttempt |
| Workload generator | `src/workload.py` | Create seeded jobs; fit oracle and pre-tuned per-class timeouts. | WorkloadSpec, seed → messages, shift time |
| Failure model | `src/failure_model.py` | Decide if an attempt crashes or hangs, and when. | seed, job id, receive count → FailureDecision |
| Ledger | `src/idempotency.py` | Commit each job's result once. | job id → True (first) / False |
| Policies | `src/policies/*.py` | Choose timeouts, heartbeat interval and budget; learn from outcomes. | job type, time, history → TimeoutDecision |
| Simulator | `src/simulator.py` | Event-driven model of one Standard SQS queue with workers. | messages, policy, failures → result dict |
| Metrics | `src/metrics.py` | Per-run metrics; aggregates and confidence intervals. | result dict → metrics row; runs → aggregates |
| Statistics | `src/stats.py` | Wilcoxon, Holm, paired comparisons, Pareto, primary tests. | run table → test tables |
| Experiment runner | `src/experiment.py` | Orchestrate everything; plots; HTML report; CLI. | config JSON → files in results/ |
| Replication harness | `src/replication.py` | Run policies on real/fake SQS with threads; simulate twin; compare. | queue URL → replication files |
| Replication report | `src/replication_report.py` | Figures and HTML for a replication run. | replication CSVs → report.html |
| AWS adapter | `src/aws_sqs_adapter.py` | Minimal opt-in boto3 wrapper (older single-job validation). | queue URL → SQS calls |

## 3.3 How components communicate

The most important connection is the **policy interface** in `src/policies/base.py`. The simulator and the replication harness talk to every policy through the same small set of methods, so any policy can be used in either without change:

| Method | Called by the simulator when… | Returns |
|---|---|---|
| `decide(job_type, now)` | a message is received | TimeoutDecision(timeout, prediction, used_fallback) |
| `heartbeat_interval_for(job_type, now)` | a message is received | seconds between heartbeats, or None |
| `max_total_visibility_for(job_type, now)` | a message is received (and at each heartbeat if the budget is dynamic) | the visibility budget |
| `extension_timeout(job_type, now)` | a heartbeat is applied | how far to push the deadline |
| `observe(job_type, duration, now)` | an attempt finishes successfully and commits | nothing (updates history) |
| `observe_attempt_end(job_type, failed, now, kind)` | any attempt ends (success, crash, hang release) | nothing (updates failure counts) |

> **Key idea**
> The simulator calls the policy; the policy never calls the simulator. Policies only see what a real worker could see: the job type, the time, and outcomes of finished attempts. This keeps policies honest and reusable on real SQS.

## 3.4 Main data structures

| Structure | Fields (main ones) | Created by / used by |
|---|---|---|
| `Message` (frozen dataclass) | job_id, job_type, processing_duration, creation_time, experiment_id, poison | workload → simulator |
| `DeliveryAttempt` | attempt_id, receipt_handle, message, worker_id, receive_time, start_time, predicted_duration, visibility_timeout, visibility_deadline, receive_count, extensions, failed, completed, hung, duplicate flags, heartbeat_interval, visibility_budget, duration | one per receive, inside the simulator |
| `TimeoutDecision` | timeout, prediction, used_fallback | policies → simulator |
| `FailureDecision` | fails, fraction (when it happens, 0.15–0.85 of the duration), hangs | failure model → simulator |
| Attempt row (dict) | 22 fields, e.g. receive_time, completion_time, assigned_visibility_timeout, duplicate_processing, committed_result | simulator → metrics, CSV |
| Result dict | attempt_rows, api_operations, failure_recoveries (+ job ids, kinds), worker_hangs, simulation_end, dead_lettered, … | simulator → metrics |

## 3.5 The life of one message

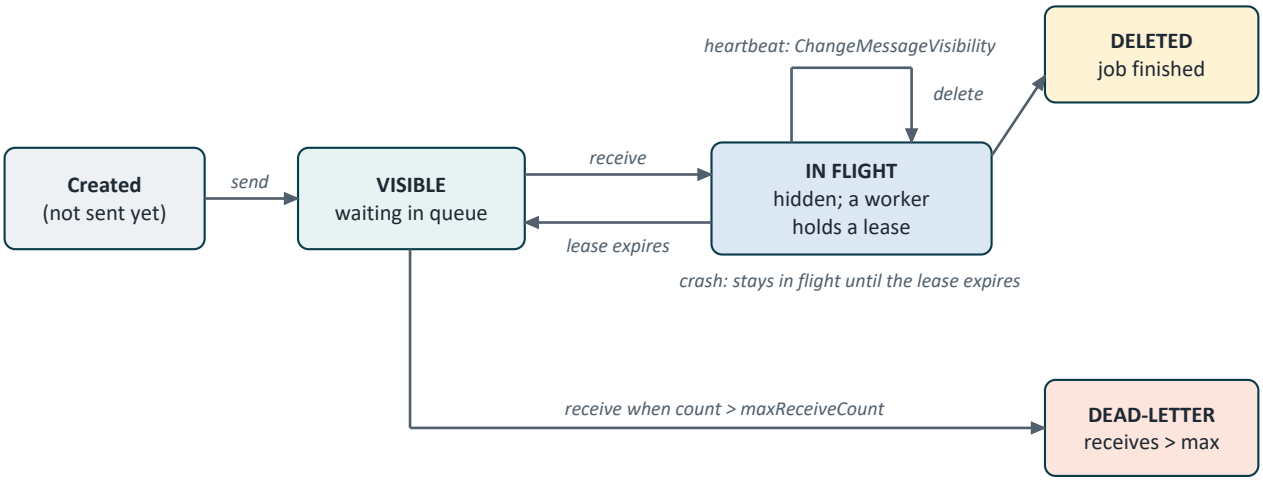



**Figure 2.** Message states in the simulator (and in SQS). A message alternates between visible and in flight until one successful attempt deletes it, or it is dead-lettered.

A message is **visible** after it is sent. A receive makes it **in flight** with a deadline. Three things can follow: a successful attempt deletes it; the deadline passes and it becomes visible again (even if the first worker is still running, which is how duplicates happen); or a heartbeat pushes the deadline later. A crash does nothing by itself: the message only reappears when its lease runs out. With a DLQ configured, a receive that would exceed maxReceiveCount moves it to the dead-letter queue instead.

## 3.6 Replication architecture (real SQS)

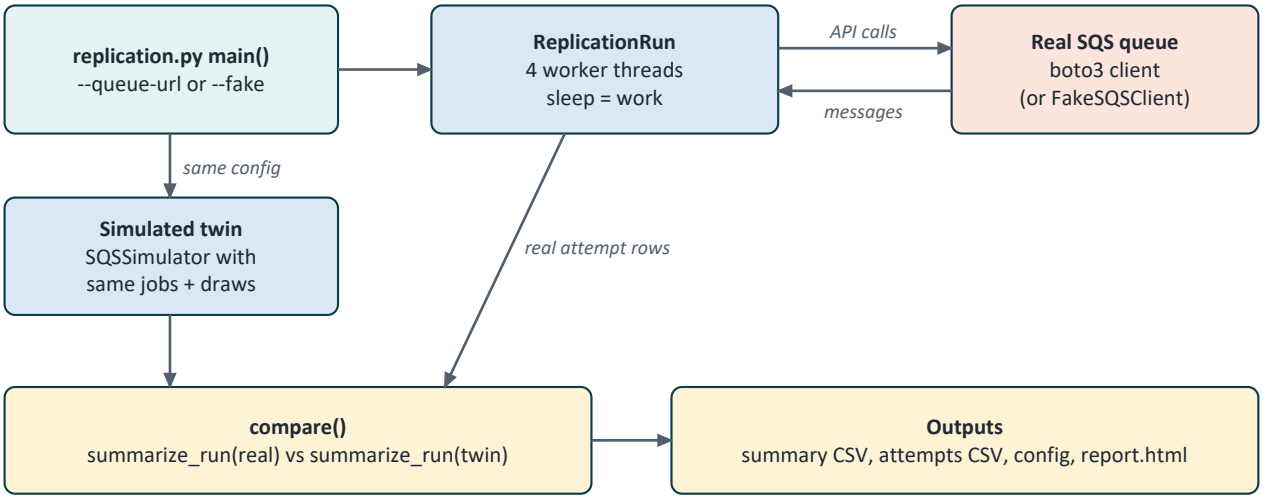



**Figure 3.** Replication harness. The same jobs and failure draws are run on a real (or fake) queue with worker threads and in the simulator, then compared metric by metric.

The harness reuses the workload generator, failure model, policy classes, simulator and metric code. It adds worker threads that really sleep for each job's duration, real API calls through boto3, an in-memory `FakeSQSClient` for offline testing, and `IntegerTimeouts`, which rounds timeouts up to whole seconds because SQS only accepts integers.

## 3.7 Technologies at a glance

| Layer | Technology | Role |
|---|---|---|
| Language | Python 3 | Everything. |
| Numerics | NumPy | Random generators, percentiles, arrays. |
| Tables | pandas | Attempt logs, metrics tables, CSV input/output, grouping. |
| Plots | matplotlib, seaborn | Figures (optional; the run still works without them). |
| Testing | unittest, pytest | 110 tests. |
| Cloud | boto3, Amazon SQS, IAM, CloudShell | Real replication run (optional). |
| Notebook | Google Colab / Jupyter | Guided run in the browser. |
| Standard library | heapq, dataclasses, collections.deque, uuid, zlib, threading, argparse, json | Event queue, data types, windows, ids, keys, threads, CLI. |

CHAPTER 4

# Complete Working and Execution Flow

This chapter follows one full experiment from the command line to the final report, without skipping steps. Section 4.6 covers errors and unusual cases, and 4.8 the replication run.

## 4.1 What happens when the project starts

```
Terminal

python -m src.experiment --config configs/full.json --output results/full --plots
```

1. Python runs `main()` in `src/experiment.py`. `argparse` reads `--config` (required), `--output` (default `results`), `--plots` and `--verbose`.
2. The JSON configuration file is read into a Python dictionary.
3. `run_experiments(config, output_dir)` is called. It creates the output folder.

## 4.2 Inputs the system receives

| Input | Example (full.json) | Effect |
|---|---|---|
| `n_messages` | 200 | Jobs per run. |
| `workers` | 10 | Number of simulated workers. |
| `arrival_rate` | 1.2 | Average jobs per second (exponential gaps). |
| `seeds` | 11, 22, …, 220 (20 seeds) | One repetition per seed. |
| `safety_margin` | 2.0 | Seconds added to percentile predictions (and to the oracle/pre-tuned fits). |
| `history_window` | 40 | Rolling window size for adaptive policies. |
| `warmup_fraction` | 0.2 | First 20% of jobs excluded from the steady_ metrics. |
| `calibration_fraction` | 0.2 | Share of the pilot workload used to pre-tune per-class timeouts. |
| `scenarios` | 28 entries | Each names a workload, failure level, optional hang level, realism options, sweeps or a policy subset. |
| `primary_hypotheses` | H1–H18 | Declared tests (scenario, reference, comparator, metric, expected direction). |

## 4.3 The experiment loop, step by step

```
Simplified from run_experiments() in src/experiment.py

for scenario in scenarios:                    # e.g. "mixed_with_failures"
    for seed in seeds:                        # 11, 22, ... 220
        messages, shift_time = generate_workload(spec, seed, experiment_id)
        realism = realism_options(scenario, messages, seed, workers)   # Phase 5 options, outages
        for margin in safety_margins:         # usually just [2.0]; a list in margin_sweep
            for window in history_windows:    # usually just [40]; a list in window_sweep
                policies = build_policies(messages, margin, window, spec, seed, ...)
                for policy in policies:
                    if a policy subset is named and this one is not in it: skip
                    sim = SQSSimulator(messages, policy, FailureModel.from_level(...),
                                       workers, seed=seed + 1000, **realism)
                    result = sim.run()
                    store attempt rows (tagged with scenario, seed, margin, window)
                    row = summarize_run(result, shift_time, warmup_fraction)
                    store row
```

Important details:

- **The workload is generated once per (scenario, seed)** and shared by every policy, so all policies see exactly the same jobs.
- **Policies are rebuilt for each (seed, margin, window)**. Adaptive policies keep history, so a fresh object means every run starts cold.
- **The simulator seed is seed + 1000**, used for failure draws (common random numbers) and worker speeds.
- **The pre-tuned baseline** is fitted to an *independent pilot workload* generated with seed + 500,000, using only its first 20% of jobs.

## 4.4 Inside one simulation run

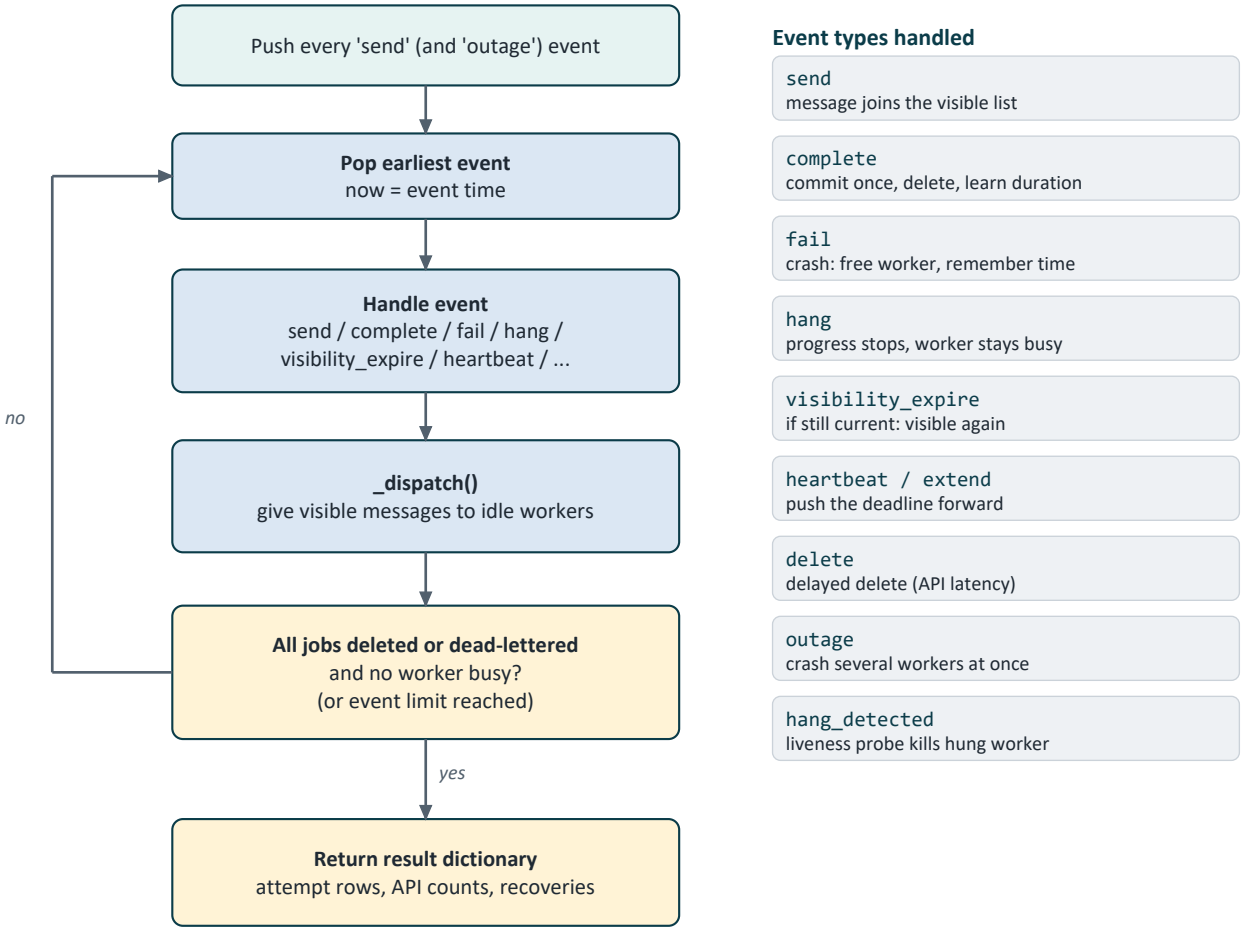



**Figure 4.** The simulator's main loop (SQSSimulator.run) and the event types it handles.

1. **Seed the event heap.** For every message, push a `send` event at its creation time. For every configured outage, push an `outage` event.
2. **Pop the earliest event** and set `now` to its time. Events at the same time come out in insertion order.
3. **Handle it.** `send`: the message joins the visible list (and the send counter increases). `complete`: commit, delete and learn (Section 5.3). `fail`: crash handling. `hang`: stop progress. `visibility_expire`: maybe make the message visible again. `heartbeat`/`extend`: maybe push the deadline. `delete`: a delayed delete. `outage`: crash running attempts on chosen workers. `hang_detected`: a liveness probe frees a hung worker.
4. **Dispatch.** After every event, `_dispatch()` hands visible messages (oldest first) to idle workers. Each receive asks the policy for a timeout, draws the attempt's failure outcome, and schedules its future events (complete or fail or hang; visibility_expire; the first heartbeat).
5. **Check for the end.** Stop when every job is deleted or dead-lettered and no worker is busy, or when a safety limit of max(10,000, 100 × jobs) events is reached (reported as `event_limit_reached`).
6. **Return** a dictionary with all attempt rows, API counts, recovery times (with job ids and kinds), hang count, end time, completed and deleted counts, dead-letter info and policy information.

## 4.5 After all runs: analysis and output

1. All attempt rows are concatenated into one table; all run rows into another (`run_df`).
2. `aggregate_runs(run_df)` groups by scenario, policy, margin and window and computes, for every numeric metric: mean, sample standard deviation, a Student-t 95% interval and a bootstrap 95% interval (2,000 resamples).
3. `_pareto_flags()` marks policies not dominated on (mean recovery, duplicate rate) within each scenario cell.
4. `paired_comparisons()` compares every policy with each reference (`adaptive_p90` and `adaptive_heartbeat`) on 7 metrics, seed by seed. All p-values in the file are Holm-adjusted together.
5. `primary_tests()` runs the 18 declared hypotheses as their own family, Holm-adjusted within it, and sets `supported` when the mean difference has the expected sign and the adjusted $p < 0.05$.
6. Five CSV files are written. With `--plots`, 12 figures are drawn. `write_html_report()` writes `report.html`.
7. The terminal prints the number of runs and attempts, the report path, a small per-policy table and the primary-hypothesis table.

## 4.6 Errors, unexpected inputs and exceptional cases

| Situation | What the code does | Where |
|---|---|---|
| Invalid parameter (negative timeout, percentile outside (0,100], heartbeat ≥ lease, window < min_history, …) | Raises `ValueError` with a message, before any simulation. | policy constructors, FailureModel, SQSSimulator |
| Unknown workload or job type | `ValueError("unknown workload")` | workload.py |
| Poison messages but no DLQ | Refuses to run: they would be redelivered forever. | SQSSimulator.__init__ |
| A run never finishes (e.g. bad settings) | Stops after the event safety limit and records `event_limit_reached = 1`. | SQSSimulator.run |
| Stale events (an old expiry after an extension, or for a deleted job) | Ignored by checking the current deadline / receipt / deleted set. | _on_visibility_expire, _apply_extension |
| Stale copy of a deleted job in the visible list | Skipped without using a worker (a Phase 1 bug fix). | _dispatch |
| No failures happened, so recovery is undefined | Metric is NaN (shown as —), never 0. | summarize_run |
| No shift observed or no convergence | Adaptation time is NaN. | summarize_run |
| matplotlib or seaborn missing | Prints a notice; CSVs and HTML report are still written. | experiment.main |
| Hypothesis or comparison file empty | Treated as an empty table. | _read_optional_csv |
| Replication: queue not empty | Raises `RuntimeError` and stops before sending anything. | run_replication |
| Replication: foreign message received | Released immediately with visibility 0. | ReplicationRun._process |
| Replication: stale receipt rejected by SQS | Caught; extension treated as failed; delete error ignored (job already committed). | _safe_change, _process |
| Replication: a policy takes too long | Stops after 600 s and records it as timed out. | ReplicationRun.run |

## 4.7 What the user ultimately sees

- In the terminal: "Completed N runs and recorded M attempts", the report path, and summary tables.
- In `results/<name>/`: `report.html` (open in a browser) with the policy comparison, primary hypotheses, headline paired comparisons and all figures, plus the five CSV files for detailed analysis.

## 4.8 The replication run, step by step

```
Terminal (real run). Use --fake for an offline dry run.

python -m src.replication --region REGION \
    --queue-url https://sqs.REGION.amazonaws.com/ACCOUNT/adaptive-visibility-test
```

1. Parse arguments; build a `ReplicationConfig` (40 jobs, 4 workers, seed 11, 8% crashes, mixed workload).
2. Create a boto3 SQS client (or the fake client with `--fake`).
3. **Safety check:** `GetQueueAttributes`; refuse unless the queue reports 0 messages.
4. Generate the 40 seeded jobs, all with send time 0, durations multiplied by `--time-scale`.
5. For each of the 4 policies (fixed_short, fixed_heartbeat, adaptive_p90, adaptive_heartbeat_dynamic):
   - Create the policy, wrapped in `IntegerTimeouts` for real runs.
   - Send all 40 messages, each tagged with a fresh run id.
   - Start 4 worker threads. Each loops: long-poll receive with a 2 s initial lease; check the run id; set the policy's lease with ChangeMessageVisibility; draw the crash outcome from the same failure model; sleep for the job (or until the crash point), sending heartbeats when due; on success, commit and delete.
   - Wait until all jobs are committed and no worker is active (or 600 s).
   - Stop the threads and delete any leftover copies of this run's messages.
   - Run the **simulated twin**: the same jobs, policy and failure seed in `SQSSimulator`.
   - Compare both with `summarize_run()`.
6. Write `replication_summary.csv`, `replication_attempts.csv`, `replication_config.json` (with `"mode": "aws"` or `"fake"`), seven figures and `report.html`; print a short table.

CHAPTER 5

# Complete Logic Explanation

This chapter explains every significant piece of logic: first the queue mechanics inside the simulator (Sections 5.1–5.8), then the policies that choose timeouts (5.9–5.10), then how results are measured and judged (5.11–5.12), and finally the real-SQS worker (5.13). Each operation uses the same structure: what, why, rules, inputs, process, output, scenarios and the reason for the design.

## 5.1 Receiving a message: _dispatch()

| Aspect | Explanation |
|---|---|
| What it does | Gives visible messages to idle workers and starts a new delivery attempt for each. |
| Why it is necessary | This is the moment the visibility timeout is chosen and all future events of the attempt are scheduled. Every duplicate and every recovery starts here. |
| Rules and conditions | Runs after every event. Workers are taken in id order; messages oldest first by original creation time. A message already deleted or dead-lettered is skipped without using a worker. If a DLQ is configured and this receive would exceed maxReceiveCount, the message is dead-lettered instead. |
| Inputs | The visible list, idle workers, receive counts, the policy, the failure model, the realism settings. |
| How it processes them | See the pseudocode below. |
| Output | New `DeliveryAttempt` objects; scheduled events; updated counters; a recovery time if the job had failed before. |
| Different scenarios | First delivery: no flags. Redelivery while the old attempt still runs: duplicate_delivery and duplicate_processing. Redelivery after a crash: duplicate_delivery only, plus a recovery time. Poison message: always scheduled to fail. |
| Why this approach | Oldest-first keeps the model simple and deterministic. Real Standard queues only promise best-effort ordering, so this is an approximation. Skipping stale copies fixed a Phase 1 bug where a worker could be stranded. |

```
Simplified from SQSSimulator._dispatch (src/simulator.py, lines 108–172)

def _dispatch(self):
    idle = [w for w in worker_ids if w not in busy]
    visible.sort(key=lambda m: (m.creation_time, m.job_id))       # oldest first
    while idle and visible:
        message = visible.pop(0)
        if job deleted or dead-lettered: continue                  # stale copy
        receive_count = receive_counts[job] + 1
        if max_receive_count and receive_count > max_receive_count:
            dead_lettered[job] = now; continue                     # SQS redrive
        worker = idle.pop(0)
        decision = policy.decide(message.job_type, now)            # <- the timeout
        attempt = DeliveryAttempt(
            receive_time=now, start_time=now + api_latency,
            visibility_deadline=now + decision.timeout,
            duplicate_delivery = receive_count > 1,
            duplicate_processing = another attempt of this job is active,
            heartbeat_interval = policy.heartbeat_interval_for(...),
            visibility_budget  = policy.max_total_visibility_for(...),
            duration = base duration x retry jitter x worker speed)
        current_receipt[job] = new uuid;  busy[worker] = attempt
        if job failed before: failure_recoveries.append(now - failure_time)
        push(deadline, "visibility_expire")
        failure = failure_model.sample_for(seed, job_id, receive_count)  # CRN
        if poison: failure = crash
        if failure.fails:   push(start + duration * fraction, "fail")
        elif failure.hangs: push(start + duration * fraction, "hang")
        else:               push(start + duration, "complete")
        if heartbeat_interval: push(now + heartbeat_interval, "heartbeat")
    api_operations["receive"] += ceil(received / receive_batch_size)
```

> **Note**
>
> The heartbeat is scheduled even for attempts that will later crash. A real worker keeps extending its lease right up to the moment it dies. Phase 3 fixed this: before the fix, doomed attempts never heartbeated, which made heartbeat policies look better at crash recovery than they are (fixed heartbeat, sudden shift: 2.4 s before, 3.5 s after).

## 5.2 Visibility expiry: _on_visibility_expire()

| Aspect | Explanation |
|---|---|
| What it does | When an attempt's deadline arrives, decides whether the message becomes visible again. |
| Why it is necessary | This is SQS's retry mechanism: it is how crashed jobs are recovered and also how duplicates are created. |
| Rules and conditions | Ignore the event if the attempt's deadline has since moved (it was extended: the old event is stale). If the attempt is hung, release the hung worker first. Do nothing if the job is deleted or dead-lettered. Make the message visible only if this attempt still holds the current receipt. |
| Inputs | The attempt, the current time, the current receipt map, deleted and dead-letter sets. |
| How it processes them | Four checks in order (see rules), then `visible.append(message)`. |
| Output | Possibly a visible message, possibly a freed (hung) worker. |
| Different scenarios | Healthy worker still running → visible → duplicate processing later. Crashed worker → visible → recovery. Hung worker → worker released and message visible. Job already done → nothing. |
| Why this approach | Checking the exact deadline (to 1e-9) is a cheap way to cancel events without removing them from the heap. |

## 5.3 Completion, commit and delete: _on_complete() and _delete()

| Aspect | Explanation |
|---|---|
| What it does | Finishes a successful attempt: frees the worker, commits the result once, deletes the message and teaches the policy. |
| Why it is necessary | This is the only place results are committed and durations are learned. |
| Rules and conditions | Ignore if already failed or completed. Commit through the ledger (first wins). Delete immediately, or after `api_latency` via a delayed `delete` event. Learn the duration (`policy.observe`) only if this attempt committed. A dead-lettered job cannot be deleted with the old handle. |
| Inputs | The attempt; the ledger; the policy. |
| How it processes them | `commit_once(job_id)` → `observe_attempt_end(failed=False)` → delete → if committed: `observe(job_type, attempt.duration)` → record row. |
| Output | Attempt row with successful_completion = True and committed_result = True or False. |
| Different scenarios | First finisher: committed and learned. Later finisher (a duplicate): rejected by the ledger, not learned, counted as duplicate_committed_result. With delete latency, a message can expire again during the delay and cause one more duplicate (tested). |
| Why this approach | Learning only from committed successes stops duplicates from counting a job's duration twice (architecture.md). Deleting on the first success is a declared simplification so runs end. |

## 5.4 Crash: _on_failure()

| Aspect | Explanation |
|---|---|
| What it does | Ends an attempt whose worker crashed part-way through. |
| Why it is necessary | Models the main failure the visibility timeout exists to handle. |
| Rules and conditions | Ignore if already completed or failed. The message is *not* made visible: it stays hidden until its lease expires. |
| Inputs | The attempt and the current time. |
| How it processes them | Mark failed, store failure time, free the worker, remove from active set, remember `last_failure[job] = now` with kind "crash", tell the policy `observe_attempt_end(failed=True)`, record the row. |
| Output | Freed worker; a pending recovery that will be measured at the next receive of this job. |
| Different scenarios | Long remaining lease → slow recovery. Heartbeat policy → the lease is short (5 s), so recovery is quick. A crash during an outage takes the same path. |
| Why this approach | Recovery time = next receive time − crash time, which is exactly what an operator would experience. |

**Closed-form check** (`test_crash_recovery_equals_timeout_minus_progress`): job of 8 s, timeout 10 s, crash at 25% (2 s). The message reappears at 10 s, so recovery = 10 − 0.25 × 8 = 8 s. The simulator gives exactly 8.0.

## 5.5 Hang and release: _on_hang(), _release_hung(), _on_hang_detected()

| Aspect | Explanation |
|---|---|
| What it does | Models a worker that stops progressing but stays alive, and frees it later. |
| Why it is necessary | Without hangs, unlimited heartbeats would be free. Hangs create the real trade-off that the visibility budget controls. |
| Rules and conditions | On hang: mark hung, record failure time and kind "hang", count it; the worker stays busy and its heartbeats continue. If a liveness probe is configured, schedule `hang_detected` after that many seconds. Release happens when the lease expires (the worker is assumed restarted then), or when the probe fires. |
| Inputs | The attempt; `hang_detection_timeout` (None or e.g. 10 s). |
| How it processes them | Release: mark failed, free worker, remove from active, `observe_attempt_end(failed=True, kind="hang")`, record row. |
| Output | A worker that is eventually freed; a hang recovery time measured at the next receive. |
| Different scenarios | Fixed heartbeat (4 extensions max): hung job hidden until about 19 s after receive. Adaptive heartbeat: until about the class's P99 × 1.25 + 2 s. Lease-only policy: until its single timeout. With the probe: heartbeats stop 10 s after the hang, then the current lease runs out. |
| Why this approach | Separating crash from hang is what makes the budget meaningful. |

**Traced example** (project code, `fixed_heartbeat`, 8 s job, hang at 50% = 4 s): heartbeats at 3.5, 7, 10.5, 14 s extend the deadline to 19 s (4 extensions, the maximum). At 19 s the worker is released, the job is received again, and it completes at 27 s. Hang recovery = 19 − 4 = **15 s**.

## 5.6 Heartbeat and extension: _on_heartbeat() and _apply_extension()

A heartbeat tries to push the attempt's deadline forward. Without API latency it is applied at once; with latency, the worker sends it now and SQS applies it `api_latency` seconds later (an `extend` event).

| Check (in order) | If true | Reason |
|---|---|---|
| Attempt failed or completed, or job deleted or dead-lettered | Stop; no more heartbeats. | Nothing to extend. |
| This attempt's receipt is no longer current | Stop. | The lease was lost to a newer receive; SQS rejects a stale handle. |
| extensions ≥ policy.max_extensions | Stop. | Extension limit reached. |
| Otherwise compute: proposed = now + extension_timeout(); if dynamic budget, re-read the budget; cap = start + budget; new = min(proposed, cap) | | |
| new deadline > current deadline | Extend: extensions + 1, change_visibility + 1, push new visibility_expire. | Only move forward. |
| Static budget and deadline has reached the cap | No further heartbeats. | A static cap can never grow. |
| Otherwise (and extensions < max) | Schedule the next heartbeat after interval − latency. | Keep renewing; latency compensation. |

With a **dynamic budget** the worker keeps heartbeating even at the cap, because the budget may grow as the policy learns that durations increased. That is the Phase 4.3 improvement tested by H10 and H11.

**Closed-form checks:** lease 2 s, heartbeat every 1 s, 7.5 s job → 7 extensions (heartbeats at t = 1…7). With max_extensions = 3 the deadline stops at 3 + 2 = 5 s and the job is redelivered at exactly 5.0 s. Both are tests in `tests/test_verification.py`.

## 5.7 Dead-letter logic

- Enabled only when `max_receive_count` is set (Phase 5 scenarios use 5; the DLQ sweep uses 2, 3, 5, 10).
- Checked at receive time: if the next receive count would exceed the maximum, the job is moved to the DLQ with the current time; any pending failure for it is forgotten (so no recovery is recorded).
- Poison messages always fail; a DLQ is mandatory when any exist.
- Metrics: `false_dlq_rate` (healthy jobs dead-lettered / healthy jobs), `dlq_but_completed` (of those, how many had already committed — replaying them would duplicate work), `poison_quarantine_time_s` (send → DLQ), `healthy_completion_rate`.

## 5.8 Correlated outages

An `outage` event crashes every attempt running on a chosen set of workers at the same moment (a hung one is released). Outage times come from an exponential process with rate `outage_rate` (0.01 per second in realistic scenarios, about one per 100 s) and each takes `outage_size` = 3 of 10 workers. Times and workers are drawn from a generator keyed by the seed, so every policy faces the same outages.

## 5.9 Policy decision logic

All policies answer the same question at receive time: *what timeout should this attempt get?* Heartbeat policies also answer *how often to renew* and *up to what budget*.

| Policy (name in results) | Timeout rule | Heartbeat | Learns? |
|---|---|---|---|
| `fixed_short` | 3 s for every job. | No | No |
| `fixed_long` | 30 s for every job. | No | No |
| `fixed_overall_mean` | Mean duration of all jobs in the evaluated workload (+0). | No | No (fitted once) |
| `fixed_per_class` | **Oracle**: per class P90 + 2 s, fitted to the evaluated workload itself; 15 s for unseen classes. | No | No |
| `pretuned_per_class` | **Realistic**: per class P90 + 2 s from the first 20% of an independent pilot workload. | No | No |
| `adaptive_p90` | Per class rolling P90 of the last 40 committed durations + 2 s, clamped to [1, 120] s; cold-start rule below 5 samples. | No | Yes |
| `fixed_heartbeat` | 5 s lease. | Every 3.5 s, max 4 extensions, budget 40 s | No |
| `adaptive_heartbeat` | min(5 s, adaptive P90 + 2 s). | Every 0.7 × lease; budget = P99 × 1.25 + 2 s (clamped to [lease, 40 s]) fixed at receive | Yes |
| `adaptive_heartbeat_dynamic` | Same as adaptive_heartbeat. | Same, but the budget is re-read at every heartbeat | Yes |
| `adaptive_heartbeat_fixed_budget` | Same lease. | Budget always the global 40 s (ablation) | Lease only |

**Adaptive variants** (each differs from `adaptive_p90` in one or two settings, so their effect can be isolated):

| Variant | Change | Purpose / result |
|---|---|---|
| `adaptive_p90_relative` | Margin = 25% of the prediction instead of 2 s. | Scale protection with duration. Not better than 2 s overall. |

| Variant | Change | Purpose / result |
|---|---|---|
| `adaptive_p90_fixed_fallback` | Old cold start: 12 s until 5 samples. | Ablation for H5 (the partial rule is better). |
| `adaptive_cost_aware` | Quantile chosen from the learned failure rate: q* = 1 − p·r/(1 − p), r = 1. | H6, H7 supported. |
| `adaptive_conformal` | Conformal order statistic instead of interpolated P90. | H8 supported (coverage ≈ 90%). |
| `adaptive_cost_conformal` | Both of the above. | Exploratory; degenerates when failures are rare. |
| `adaptive_p90_cpd` | Page–Hinkley change detection, threshold 3. | H9 supported (faster reaction to shifts). |
| `adaptive_p90_cpd_scaled` | Threshold scaled by the class's spread / 0.25. | H17, H18 supported (fewer false alarms). |
| `adaptive_heartbeat_cpd_dynamic` | Dynamic heartbeat + change detection. | Exploratory; little gain. |
| `adaptive_heartbeat_cost_budget` | Budget quantile from the hang rate, r = 0.5. | Rejected at tuning; kept as exploratory. |
| Sweep names | `adaptive_p75`…`adaptive_p99`, `adaptive_h2/h10/h20`, `adaptive_cost_r0.25/r4`, `adaptive_heartbeat_b1.0/b1.5/b2.0` | Sensitivity sweeps for percentile, min_history, cost ratio and budget factor. |

## The adaptive controller's decision, as a decision table

| Samples n for this class | cold_start setting | Timeout | used_fallback |
|---|---|---|---|
| 0 | any | default_timeout (12 s in experiments; 5 s lease for heartbeat policies) | True |
| 1 to 4 | "partial" (default) | clamp(1.5 × max(samples) + margin) | True |
| 1 to 4 | "default" | default_timeout | True |
| ≥ 5 | quantile_method "interpolate" | clamp(percentile(samples, 100q) + margin) | False |
| ≥ 5 | quantile_method "conformal" | clamp(sorted[ceil((n+1)q) − 1] + margin) if that rank exists, else the cold-start rule | False / True |

Here q is 0.9 (P90) unless `cost_ratio` is set, in which case q is chosen from the failure rate. The margin is `safety_margin + relative_margin × prediction + mad_multiplier × MAD` (only the 2 s absolute part is used in the main experiments). Clamp means min(120, max(1, value)). For `AdaptiveHeartbeatPolicy` the result is then capped at the 5 s lease.

`AdaptivePercentilePolicy.decide (src/policies/adaptive_percentile.py)`

```
def decide(self, job_type, now):
    values = self.history[job_type]                      # deque, newest last, at most W values
    prediction = self._quantile(values) if len(values) >= self.min_history else None
    if prediction is None:
        return self._cold_start_decision(values)         # partial or default rule
    timeout = clamp(prediction + self.margin(samples, prediction))
    return TimeoutDecision(timeout, prediction, used_fallback=False)
```

## 5.10 Learning from outcomes

| Callback | When | What the adaptive policy does |
|---|---|---|
| `observe(job_type, duration, now)` | After a committed success. | Append the duration to the class window. If change detection is on, update the Page–Hinkley statistics on log(duration); on an alarm, cut the window to its last 3 values and count a change point. |
| `observe_attempt_end(job_type, failed, now, kind)` | After any attempt ends. | attempts_seen += 1; failures_seen += failed. The heartbeat policy also counts hangs. These feed the Beta-prior failure and hang rates used by cost-aware rules. |

> **Note**
> Static policies inherit empty versions of these callbacks, so the simulator can call them on every policy.

## 5.11 Measuring a run: summarize_run()

After each simulation, `summarize_run()` in `src/metrics.py` turns the attempt rows into one row of metrics. The most important ones:

| Metric | Computation |
|---|---|
| `duplicate_processing_rate` | Mean of the duplicate_processing flag over **all attempts** (so the denominator is attempts, not jobs). |
| `mean_failure_recovery_time_s` | Mean of (next receive − failure time) over all recoveries; NaN if none. |
| `mean_end_to_end_latency_s` | Mean of (completion − send) over committed attempts. |
| `throughput_jobs_per_s` | Completed jobs / simulation end time. |
| `wasted_processing_time_s` | Work done by crashed attempts + time of successful attempts flagged as duplicate processing. |
| `mean_absolute_prediction_error_s` | Mean \|actual − predicted\| over successful attempts; only for policies that predict durations. |
| `prediction_coverage` | Share of first, non-fallback successful attempts with actual ≤ predicted quantile (before margin). |
| `tradeoff_cost_s_per_job` | (seconds of duplicate-processing work + total recovery seconds) / completed jobs. |
| `queue_api_operations_per_completed_message` | All send, receive, delete and change-visibility calls / completed jobs. |
| `adaptation_time_s` | After a shift: time until 5 consecutive predictions are within 20% (min 0.5 s) of the post-shift P90. |
| `steady_*` | Five key metrics recomputed for jobs sent after the first 20% (warm-up excluded). |
| DLQ metrics | false_dlq_rate, dlq_but_completed, healthy_completion_rate, poison_quarantine_time_s (Section 5.7). |

## 5.12 Judging results: when is a hypothesis "supported"?

1. Take the runs of the declared scenario at the default margin and window.
2. Match the reference and comparator policies seed by seed; compute differences d = reference − comparator.
3. Run the two-sided Wilcoxon signed-rank test on d (zero differences dropped).
4. Holm-adjust the 18 p-values together.
5. `supported = (mean d has the expected sign) and (Holm-adjusted p < 0.05)`.

Everything else (`paired_comparisons.csv`) is **exploratory** and Holm-adjusted across all its rows, which makes it much stricter. The analysis rules in `docs/methodology.md` say to read the primary family first.

## 5.13 The real-SQS worker: ReplicationRun._process()

```
Simplified from src/replication.py, lines 252–318

receive with a 2 s lease (job type unknown until the body is read)
if body.run_id != my run id: set visibility 0 (release it untouched); return
decide timeout, heartbeat interval, budget (under a lock); note duplicate if job active
if job crashed earlier: recovery = now - crash time
ChangeMessageVisibility(receipt, policy timeout)           # one extra call per receive
failure = failure_model.sample_for(seed, job_id, ApproximateReceiveCount)
work = duration x (crash fraction if it will crash else 1)
while elapsed < work:
    sleep up to 20 ms
    if heartbeat due and extensions < max:
        new_deadline = min(now + extension, start + budget)    # budget re-read if dynamic
        if new_deadline > deadline and ChangeMessageVisibility succeeds: extensions += 1
if crashed: remember crash time; tell policy (failed)       # message left to expire
else: commit once; tell policy; learn duration if first; DeleteMessage
record an attempt row
```

**Differences from the simulator** (documented in docs/aws_replication.md): the initial 2 s lease plus one extra ChangeMessageVisibility per receive; a crash is simulated inside a live thread (the thread just stops working on the job); network latency, long-poll timing and thread scheduling are real; hangs, DLQ and outages are not used.

CHAPTER 6

# Source Code Explanation

## 6.1 Folder structure

```
Repository layout (commit 4730bfb)

adaptive-sqs-controller/
  src/                          the Python package
    message.py                  Message and DeliveryAttempt data classes
    workload.py                 seeded job generator; oracle and pre-tuned per-class timeouts
    failure_model.py            crash/hang decisions with common random numbers
    idempotency.py              first-writer-wins ledger
    simulator.py                SQSSimulator: the discrete-event queue model
    metrics.py                  per-run metrics, aggregates, Student-t critical values
    stats.py                    Wilcoxon, Holm, paired comparisons, Pareto, primary tests
    experiment.py               experiment runner, policy factory, plots, HTML report, CLI
    replication.py              real/fake SQS replication harness and CLI
    replication_report.py       figures and HTML for a replication run
    aws_sqs_adapter.py          small boto3 adapter (older, single-job validation)
    policies/
      base.py                   VisibilityPolicy interface, TimeoutDecision
      fixed.py                  FixedPolicy, MeanFixedPolicy
      class_based.py            ClassBasedPolicy (oracle and pre-tuned)
      heartbeat.py              FixedHeartbeatPolicy
      adaptive_percentile.py    AdaptivePercentilePolicy (+ cold start, conformal, cost, PH)
      adaptive_heartbeat.py     AdaptiveHeartbeatPolicy (lease, interval, budget)
  tests/                        110 tests (unittest + pytest style)
  configs/smoke.json, full.json
  docs/                         architecture, methodology, limitations, AWS guide, IAM policy
  results/smoke/, full/, aws_replication/
  scripts/generate_plots_windows.ps1   plot fallback without matplotlib
  Adaptive_SQS_Controller_Colab.ipynb  guided notebook
  presentation/, Adaptive_SQS_Controller_Explanation.docx, README.md, requirements.txt
```

## 6.2 Data model and inputs

### src/message.py

`Message` is a **frozen** dataclass (cannot be changed after creation, so it is safe to share between policies and runs). `DeliveryAttempt` is mutable: it is updated as the attempt is extended, completes or fails. Its `duration` field is the attempt's own processing time (base duration × jitter × worker speed), which can differ between attempts of the same job when Phase 5 realism is on.

### src/workload.py

#### `generate_workload(spec, seed, experiment_id)`

| | |
|---|---|
| **Purpose** | Create a reproducible list of jobs for one scenario and seed. |
| **Inputs and parameters** | • `spec`: WorkloadSpec (name, n_messages, arrival_rate, burst_probability, burst_size, shift_fraction, poison_fraction).<br>• `seed`: integer; `experiment_id`: label stored on each message. |

| | |
|---|---|
| **Internal logic, step by step** | • Create `np.random.default_rng(seed)`.<br>• For each index: gap = 0 with burst probability, else exponential(1/arrival_rate); now += gap.<br>• Choose job type by workload: mostly_short 90/10 short/medium; mixed 50/30/20 short/medium/long; heavy_tailed all heavy; bursty groups of burst_size share a send time; gradual_shift moves from 70% short to 60% long; sudden_shift all medium.<br>• For sudden_shift, jobs from index n × shift_fraction onward are "shifted" (durations × 2); the first shifted send time is the shift time.<br>• Draw the duration with `_duration()` (log-normal or Pareto, clipped to [0.15, 90]).<br>• If poison_fraction > 0, mark poison jobs with a *separate* generator so all other draws stay identical. |
| **Important conditions and edge cases** | Unknown workload or job type raises ValueError. Bursts and the bursty pattern can make several jobs share a send time. |
| **Output or side effects** | (list of Message, shift_time or None). |
| **How other parts use it** | run_experiments (once per scenario and seed), pretuned_class_timeouts (pilot), replication (scaled_workload). |
| **Worked example** | mixed, seed 11, first jobs (computed): job-00000 short 1.57 s sent 0.19; job-00001 long 16.80 s sent 0.29; job-00002 short 1.17 s; job-00004 long 17.44 s; job-00005 medium 4.14 s. |

## `pretuned_class_timeouts(spec, seed, margin=2.0, calibration_fraction=0.2)`

| | |
|---|---|
| **Purpose** | Build the **realistic** static baseline: what an operator would configure after watching a calibration period. |
| **Inputs and parameters** | • spec, seed of the evaluated run; margin; calibration fraction in (0, 1]. |
| **Internal logic, step by step** | • Generate a pilot workload with seed + 500,000 (never the evaluated jobs).<br>• Keep only its first 20% of jobs.<br>• Per class: P90 of durations + margin (`oracle_class_timeouts` applied to the calibration slice). |
| **Important conditions and edge cases** | Classes missing from the calibration slice fall back to the policy default (15 s). After a shift, the timeouts are stale by design. |
| **Output or side effects** | dict job_type → timeout. |
| **How other parts use it** | build_policies creates `ClassBasedPolicy(..., name="pretuned_per_class")`. |
| **Worked example** | mixed, seed 11 (computed): pre-tuned short 3.35, medium 9.08, long 20.08 s; oracle (whole evaluated workload) 3.56, 9.21, 21.75 s. sudden_shift: pre-tuned medium 8.4 s, but after the shift medium jobs have a median of 10 s. |

### src/failure_model.py

## `FailureModel.sample_for(seed, job_id, attempt_index)`

| | |
|---|---|
| **Purpose** | Decide whether this delivery crashes or hangs, and at what fraction of its duration — identically under every policy. |
| **Inputs and parameters** | • seed (simulator seed), job_id, attempt_index (= receive count). |
| **Internal logic, step by step** | • key = [seed, crc32(job_id), attempt_index]; rng = default_rng(key).<br>• fails = rng.random() < crash probability.<br>• fraction = uniform(0.15, 0.85) (never at the very start or end, so wasted work is measurable).<br>• hangs = rng.random() < hang probability and not fails (drawn last, so adding hangs leaves crash draws unchanged). |
| **Important conditions and edge cases** | Probabilities must be in [0, 1). Crash and hang are mutually exclusive. |
| **Output or side effects** | FailureDecision(fails, fraction, hangs). |
| **How other parts use it** | SQSSimulator._dispatch and ReplicationRun._process (the real run uses the same draws as its twin). |
| **Worked example** | seed 11, job-00003, 8% crashes: attempts 1, 2, 3 → no crash, fractions 0.80, 0.38, 0.74 (computed). |

### src/idempotency.py

`commit_once(job_id)` takes a `threading.Lock`, returns False if the id is already in the set, otherwise adds it and returns True. The lock matters in the threaded replication harness; in the single-threaded simulator it is harmless. `len(ledger)` is the number of completed jobs.

## 6.3 Policies (src/policies/)

`VisibilityPolicy` (base.py) is an abstract class. Only `decide()` is required; the other methods have defaults (no learning, no heartbeat, budget = `max_total_visibility`). `clone()` deep-copies a policy. `predicts_duration` is False for static policies so their timeout is not mistaken for a prediction.

```
fixed.py, class_based.py, heartbeat.py (abridged)

class FixedPolicy(VisibilityPolicy):
    def decide(self, job_type, now):
        return TimeoutDecision(self.timeout, self.timeout)      # same for everyone

class ClassBasedPolicy(VisibilityPolicy):
    def decide(self, job_type, now):
        fallback = job_type not in self.timeouts
        timeout = self.timeouts.get(job_type, self.default_timeout)
        return TimeoutDecision(timeout, timeout, fallback)

class FixedHeartbeatPolicy(FixedPolicy):                        # adds heartbeat settings only
    # heartbeat_interval < timeout is enforced; max_extensions; max_total_visibility
```

### AdaptivePercentilePolicy.decide(job_type, now)

| | |
|---|---|
| **Purpose** | Choose the timeout for this class from its recent successful durations. |
| **Inputs and parameters** | • job_type; now (unused by the basic rule). State: `self.history[job_type]` (a deque of at most W values). |
| **Internal logic, step by step** | • If the class has ≥ min_history samples, compute the prediction with `_quantile()` (interpolated or conformal, at level `target_quantile()`).<br>• If there is no prediction (too few samples, or the conformal rank does not exist), return `_cold_start_decision()`.<br>• Otherwise timeout = clamp(prediction + margin(samples, prediction)) to [min_timeout, max_timeout]. |
| **Important conditions and edge cases** | Empty history → default timeout. Partial history → 1.5 × max + margin. The deque drops old values automatically. |
| **Output or side effects** | TimeoutDecision(timeout, prediction, used_fallback). |
| **How other parts use it** | Simulator and replication on every receive; AdaptiveHeartbeatPolicy.decide caps it at the lease. |
| **Worked example** | History of class long: 13.1, 15.8, 12.4, 17.2, 14.0 → P90 = 16.64 → timeout 18.64 s (computed; see Example 3). |

### AdaptivePercentilePolicy._page_hinkley(job_type, x)

| | |
|---|---|
| **Purpose** | Detect a sustained rise or fall in a class's log-duration, so old history can be dropped. |
| **Inputs and parameters** | • x = log(duration); per-class state [n, mean, up, min_up, down, max_down]; ph_delta (0.05), ph_lambda (3 in experiments), optional ph_spread_reference (0.25). |

| | |
|---|---|
| **Internal logic, step by step** | • n += 1; mean += (x − mean)/n (running mean).<br>• up += x − mean − δ; down += x − mean + δ.<br>• min_up = min(min_up, up); max_down = max(max_down, down).<br>• threshold = λ, multiplied by max(1, std(log history)/reference) if spread scaling is on and history ≥ min_history.<br>• Alarm if up − min_up > threshold (rise) or max_down − down > threshold (fall): reset state, return True. |
| **Important conditions and edge cases** | δ is a drift allowance that stops slow noise from accumulating. Log scale makes a doubling the same size for every class. |
| **Output or side effects** | True on alarm. Then `observe()` keeps only the last 3 durations and increments `change_points`. |
| **How other parts use it** | observe() when change_detection is on. |
| **Worked example** | Medium durations doubled after 20 samples: alarm on the 6th post-shift sample, when up − min_up passed 3; window cut from 26 to 3 (computed; Example 5). |

### `AdaptiveHeartbeatPolicy.max_total_visibility_for(job_type, now)`

| | |
|---|---|
| **Purpose** | Return the visibility budget: how long, from processing start, this attempt may stay hidden in total. |
| **Inputs and parameters** | • job_type; class history; budget_percentile (99), budget_factor (1.25), budget_margin (2), max_total_visibility (40 in experiments); optional budget_cost_ratio. |
| **Internal logic, step by step** | • If the adaptive budget is off, or fewer than min_history samples: return the global cap.<br>• Else budget = P99(history) × 1.25 + 2 (or, cost-aware: quantile q* of history + 2).<br>• Return min(global cap, max(lease, budget)). |
| **Important conditions and edge cases** | Never below one lease, never above the global cap. |
| **Output or side effects** | Seconds. |
| **How other parts use it** | Simulator at receive; again at each heartbeat when `dynamic_budget` is True. |
| **Worked example** | Class long with history 13.1 … 13.7 (P99 = 19.36): budget = 19.36 × 1.25 + 2 = 26.2 s. Class short with P99 1.40: 3.75 s (computed). |

`AdaptiveHeartbeatPolicy` also overrides `decide()` (cap at `lease_cap` = 5 s), `heartbeat_interval_for()` (0.7 × lease), and `observe_attempt_end()` (counts hangs for the cost-aware budget). Its `max_extensions` is 1,000, so in practice the budget, not the count, stops renewals.

## 6.4 The simulator (src/simulator.py)

| Method | Lines | Role |
|---|---|---|
| `__init__` | 43–103 | Validate realism options; set worker speeds (seeded); create ledger, heap, state maps, counters. |
| `_push` | 105 | heappush((time, counter, event, payload)) — the counter makes ties deterministic. |
| `_dispatch` | 108–172 | Start attempts on idle workers (Section 5.1). |
| `_attempt_duration` | 174–181 | Base duration × seeded log-normal retry jitter (mean 1) × worker speed. |
| `_record` | 183–212 | Append one attempt row (22 fields). |
| `_on_failure` | 214–224 | Crash (Section 5.4). |
| `_on_hang / _release_hung / _on_hang_detected` | 226–250 | Hang lifecycle (Section 5.5). |
| `_on_complete / _delete` | 252–275 | Commit, delete, learn (Section 5.3). |
| `_on_outage` | 277–286 | Correlated crash of chosen workers. |

| Method | Lines | Role |
|---|---|---|
| `_on_visibility_expire` | 288–297 | Re-show the message if appropriate (Section 5.2). |
| `_on_heartbeat / _apply_extension` | 299–334 | Extend leases (Section 5.6). |
| `run` | 336–393 | Main loop and result dictionary. |

### `SQSSimulator._apply_extension(attempt, next_heartbeat_delay)`

| | |
|---|---|
| **Purpose** | Apply one heartbeat: push the deadline forward within the budget, and schedule the next heartbeat. |
| **Inputs and parameters** | • attempt; next_heartbeat_delay (0 normally; the API latency when called from a delayed `extend` event). |
| **Internal logic, step by step** | • Return if the attempt is finished or the job deleted/dead-lettered.<br>• Return if the receipt is no longer current, or the extension limit is reached.<br>• proposed = now + policy.extension_timeout(); if dynamic, re-read the budget; cap = start + budget; new = min(proposed, cap).<br>• If new > current deadline: count the extension and the API call, set the deadline, push a new visibility_expire.<br>• Unless a static cap was reached or the limit hit, push the next heartbeat at now + interval − next_heartbeat_delay. |
| **Important conditions and edge cases** | Old visibility_expire events stay in the heap and are ignored later because the deadline changed. A hung attempt passes all checks, so it keeps extending until the cap. |
| **Output or side effects** | Mutates the attempt; increments `api_operations["change_visibility"]`; pushes events. |
| **How other parts use it** | Called from _on_heartbeat (no latency) or from the extend event (with latency). |
| **Worked example** | Fixed heartbeat, 8 s job: heartbeats at 3.5 and 7 s; deadlines 8.5 then 12 s; the job completes at 8 s (Example 2 shows 1 extension for a 4 s job and 1 for a 7 s job). |

## 6.5 Metrics and statistics

### `summarize_run(sim_result, shift_time=None, warmup_fraction=0.2)`

| | |
|---|---|
| **Purpose** | Turn one run's attempt rows into a dictionary of about 35 metrics. |
| **Inputs and parameters** | • The simulator's (or replication's) result dict; the shift time for adaptation; the warm-up fraction. |
| **Internal logic, step by step** | • Build a DataFrame of attempts; select committed and successful rows.<br>• Compute duplicate rates (mean of flags), latency (completion − send), throughput, recovery statistics (all and hang-only), prediction error and coverage (only for predicting policies), wasted time, trade-off cost, API calls per job, adaptation time.<br>• Add DLQ metrics (`_dead_letter_metrics`) and steady-state metrics (`_steady_state_metrics`). |
| **Important conditions and edge cases** | Missing quantities become NaN (not 0). Static policies get no prediction metrics. Coverage uses first attempts only. |
| **Output or side effects** | dict of floats. |
| **How other parts use it** | run_experiments (one row per run) and replication.compare (real and twin). |
| **Worked example** | Example 1's three-job run: 6 attempts, duplicate rate 3/6 = 0.5, latency 4.07 s, wasted 18 s, 4 API calls per job (computed). |

## t_critical(confidence, df) and aggregate_runs(run_metrics)

| | |
|---|---|
| **Purpose** | Give honest 95% intervals across seeds without SciPy. |
| **Inputs and parameters** | • confidence (0.95), degrees of freedom n − 1; the run table. |
| **Internal logic, step by step** | • t_critical: bisection on x in (0, 1) using the regularized incomplete beta function I_x(df/2, 1/2) (computed with a continued fraction, `_betacf`/`_betainc`), then t = sqrt(df(1 − x)/x).<br>• aggregate_runs: group by scenario, policy, margin, window; for each metric: mean, sample SD, mean ± t·SD/√n, and a percentile bootstrap (2,000 resamples of the seeds, seed 0). |
| **Important conditions and edge cases** | One seed → zero-width interval. The docstring notes t(0.975, 1) = 12.7, far wider than the 1.96 the first version used. |
| **Output or side effects** | DataFrame with *_mean, *_std, *_ci95_low/high, *_boot_ci95_low/high. |
| **How other parts use it** | run_experiments. |
| **Worked example** | With 20 seeds, df = 19, t ≈ 2.09. |

## wilcoxon_signed_rank(differences)

| | |
|---|---|
| **Purpose** | Two-sided test of whether paired differences are centred on zero. |
| **Inputs and parameters** | • A list of per-seed differences. |
| **Internal logic, step by step** | • Drop NaN and zero differences; n = remaining count.<br>• Rank \|d\| (ties share the average rank); W+ = sum of ranks of positive differences.<br>• No ties and n ≤ 50: exact null distribution of W+ by dynamic programming (count the subsets of ranks 1…n with each sum), p = 2 × min(lower tail, upper tail).<br>• Otherwise: normal approximation with tie correction and continuity correction. |
| **Important conditions and edge cases** | All zero → p = NaN. Smallest exact p with 20 seeds is 2/2^20 ≈ 1.9 × 10^-6. |
| **Output or side effects** | (W+, p, n used). |
| **How other parts use it** | paired_comparisons and primary_tests. |
| **Worked example** | All 20 differences negative → W+ = 0 → p ≈ 1.9 × 10^-6 before Holm. |

## holm_adjust(p_values)

| | |
|---|---|
| **Purpose** | Correct a family of p-values for multiple testing. |
| **Inputs and parameters** | • List of p-values (NaN allowed). |
| **Internal logic, step by step** | • Sort valid p-values ascending: p(1) ≤ … ≤ p(m).<br>• adjusted(k) = max over j ≤ k of min(1, (m − j + 1) × p(j)) (running maximum keeps order). |
| **Important conditions and edge cases** | NaNs are passed through untouched. |
| **Output or side effects** | Adjusted p-values in the original order. |
| **How other parts use it** | primary_tests (18-test family) and run_experiments (whole exploratory file). |
| **Worked example** | p = 0.01, 0.02, 0.04 → 0.03, 0.04, 0.04. |

`paired_comparisons()` matches seeds between a reference and each other policy within each scenario cell and reports the mean difference, a t interval, how many seeds favour the reference, W+ and p. `pareto_optimal()` flags rows not dominated on two lower-is-better axes. `primary_tests()` runs the declared family (Section 5.12).

## 6.6 Experiment runner (src/experiment.py)

| Function | What it does |
|---|---|
| `DEFAULT_SCENARIOS`, `DEFAULT_ADAPTIVE_VARIANTS` | Fallback scenarios and the adaptive variants run alongside adaptive_p90. |
| `build_policies()` | Creates fixed_short (3 s), fixed_long (30 s), fixed_overall_mean, oracle, pre-tuned, adaptive_p90 (default 12 s, min_history 5), adaptive_heartbeat, its fixed-budget ablation, every variant, and fixed_heartbeat (5 s/3.5 s/4/40 s). |
| `realism_options()` | Collects Phase 5 settings from a scenario and generates seeded outages. |
| `run_experiments()` | The main loop (Chapter 4); writes the five CSVs. |
| `_pareto_flags()` | Pareto flag per scenario cell on the aggregates. |
| `generate_plots()` | 12 figures (duplicates, recovery, latency, throughput, margin sweeps, prediction vs actual, shift, API overhead, trade-off, Pareto front, cold start vs steady). |
| `write_html_report()` | Builds report.html from the CSVs and figures. |
| `main()` | Command-line entry point. |

> **Note**
> Variant names drive behaviour: an override dictionary with `"type": "heartbeat"` builds an `AdaptiveHeartbeatPolicy`; otherwise an `AdaptivePercentilePolicy`. The history window is raised to min_history if a sweep would make it smaller.

## 6.7 Replication (src/replication.py, replication_report.py, aws_sqs_adapter.py)

| Piece | What it does |
|---|---|
| `FakeSQSClient` | In-memory queue following SQS visibility rules with wall-clock time: ChangeMessageVisibility needs the latest handle (else InvalidReceiptHandle); DeleteMessage accepts any handle of the message. |
| `IntegerTimeouts` | Wraps a policy and rounds every timeout and extension up to whole seconds in [1, 43200] (SQS limits). Used for the real run and its twin. |
| `replication_policies(time_scale)` | The 4 compared policies with fixed values scaled by time_scale. |
| `ReplicationRun` | One policy's real run: send, worker threads, heartbeats, crashes, rows, drain leftovers. |
| `scaled_workload / simulate_twin / compare` | Same jobs at time 0; the simulator twin; the metric-by-metric comparison. |
| `run_replication / main` | Safety check, loop over policies, write files, open the report. |
| `replication_report.py` | 7 figures (6 metrics real vs simulated + attempt timeline) and an HTML page with a FAKE/REAL banner. |
| `aws_sqs_adapter.py` | Earlier minimal adapter (send, receive, extend, delete, validate one job). Not used by the replication harness. |

## 6.8 How the functions interact: a call trace

```
Execution trace through the code

experiment.main
 └─ run_experiments(config)
     ├─ generate_workload(spec, seed)                          workload.py
     ├─ realism_options(...)
     ├─ build_policies(messages, margin, window, spec, seed)
     │    ├─ oracle_class_timeouts(messages)                   workload.py
     │    └─ pretuned_class_timeouts(spec, seed)  → generate_workload(seed + 500000)
     ├─ SQSSimulator(messages, policy, FailureModel.from_level(...)).run()
     │    └─ loop: heappop → _on_* handlers → _dispatch
     │         ├─ policy.decide / heartbeat_interval_for / max_total_visibility_for
     │         ├─ failure_model.sample_for(seed, job, k)
     │         ├─ ledger.commit_once → policy.observe / observe_attempt_end
     │         └─ _apply_extension → policy.extension_timeout
     ├─ summarize_run(result)                                  metrics.py
     ├─ aggregate_runs → t_critical                            metrics.py
     ├─ paired_comparisons / primary_tests → wilcoxon_signed_rank, holm_adjust   stats.py
     └─ to_csv x5
 ├─ generate_plots (optional)
 └─ write_html_report
```

**Key idea**

Architecture to code mapping: the boxes in Figure 1 are exactly these modules. The policy interface (base.py) is the seam between "how SQS behaves" (simulator) and "how to choose a timeout" (policies).

CHAPTER 7

# Algorithms and Mathematical Logic

## 7.1 What is implemented and what is only discussed

| Algorithm / formula | Status | Where |
|---|---|---|
| Rolling per-class percentile + margin | Implemented, main controller | adaptive_percentile.py |
| Partial-history cold start (1.5 × max + margin) | Implemented, default | adaptive_percentile.py |
| Relative and MAD margins | Implemented; relative tested, MAD only unit-tested | adaptive_percentile.py |
| Conformal quantile | Implemented (variant) | adaptive_percentile.py |
| Cost-aware quantile q* with Beta prior | Implemented (variant) | adaptive_percentile.py |
| Two-sided Page–Hinkley change detection (+ spread scaling) | Implemented (variants) | adaptive_percentile.py |
| Adaptive lease, heartbeat interval, P99 budget, dynamic budget | Implemented | adaptive_heartbeat.py, simulator.py |
| Cost-aware heartbeat budget | Implemented, rejected at tuning, kept exploratory | adaptive_heartbeat.py |
| Discrete-event simulation with a binary heap | Implemented | simulator.py |
| Common random numbers via keyed generators | Implemented | failure_model.py, simulator.py |
| Student-t interval (incomplete beta), bootstrap, Wilcoxon (exact DP), Holm, Pareto | Implemented without SciPy | metrics.py, stats.py |
| Erlang C (M/M/c mean wait) | Implemented in a test only, as a check | tests/test_verification.py |
| EWMA or streaming quantile sketches | Proposed (README future work) | — |
| Machine-learning duration prediction | Not used; discussed as an alternative (README viva notes) | — |

## 7.2 Percentile with linear interpolation

**Intuition:** sort the values; P90 is the point 90% of the way along the sorted list, interpolating between neighbours. **Formula** (NumPy's default): with n sorted values v[0..n−1], position h = (n − 1) × q; P = v[floor(h)] + (h − floor(h)) × (v[floor(h)+1] − v[floor(h)]).

**Worked example:** 12.4, 13.1, 14.0, 15.8, 17.2 and q = 0.9. h = 4 × 0.9 = 3.6. P90 = 15.8 + 0.6 × (17.2 − 15.8) = 15.8 + 0.84 = **16.64**. The code returns 16.64 (Example 3). **Cost:** sorting W ≤ 40 values, O(W log W) per receive.

## 7.3 The timeout formula

```
timeout = min(T_max, max(T_min, P_q(history[class]) + margin))
margin  = safety_margin + relative_margin x prediction + mad_multiplier x MAD(history)
```

| Symbol | Meaning | Value in the main experiments |
|---|---|---|
| P_q | q-quantile of recent committed durations of the class | q = 0.9 |
| history | last W committed durations of the class | W = 40 |
| safety_margin | fixed seconds added | 2 s (margin sweep 0, 1, 2, 4) |
| T_min, T_max | clamp limits | 1 s, 120 s |
| min_history | samples needed before using P_q | 5 (sweep 2, 5, 10, 20) |

**Effect:** a larger q or margin means fewer overruns (fewer duplicates) but a longer wait after a crash. The percentile and margin sweeps traced exactly this smooth curve (README, Phase 2).

## 7.4 Partial-history cold start

**Intuition:** with n samples from a distribution, the chance that the next value is larger than all of them is 1/(n + 1) (for exchangeable data). So the maximum is already a decent upper guess; multiplying by 1.5 widens it for the tail. **Rule:** for 1 ≤ n < min_history, prediction = 1.5 × max(samples); timeout = clamp(prediction + margin).

**Example** (class long, computed): after one sample of 13.1 s the timeout is 1.5 × 13.1 + 2 = 21.65 s. The old fixed fallback would give 12 s, shorter than the job, guaranteeing a duplicate. The factor 1.5 was chosen on tuning seeds 1001–1010.

## 7.5 Conformal quantile

**Intuition:** instead of interpolating, take a specific sorted sample chosen so that, if recent durations are exchangeable, the next duration is at or below it with probability at least q. **Rule:** rank r = ceil((n + 1) × q); prediction = the r-th smallest sample; if r > n, fall back to the cold-start rule.

**Example:** n = 40, q = 0.9 → r = ceil(36.9) = 37, the 37th smallest of 40. Interpolated P90 sits at position 35.1, slightly lower, which is why it covers less. **Measured coverage** (first attempts, before margin): mixed with failures 90.8% conformal vs 86.7% interpolated; heavy-tailed 89.2% vs 86.5%; after a shift 82.6% vs 79.8% (results/full). **Limitation:** with W = 40, levels above 40/41 ≈ 0.976 have no rank, so `adaptive_cost_conformal` falls back to cold start whenever the cost-aware level is that high.

## 7.6 Cost-aware quantile

**Intuition:** the right percentile depends on how often workers fail. If crashes are rare, waiting longer costs little, so use a higher percentile; if they are common, a shorter timeout pays off. **Derivation** (from the code docstring): with crash probability p, cost per second of overrun c_dup and of recovery delay c_rec, the expected cost is minimised where P(D > T) = p·c_rec / ((1 − p)·c_dup). So:

```
q* = 1 - p * r / (1 - p),   r = c_rec / c_dup,   q* clamped to [0.5, 0.995]
p = (failures_seen + 1) / (attempts_seen + 1 + 19)      # Beta(1, 19) prior, mean 5%
```

| Situation | p | q* |
|---|---|---|
| Before any data (prior) | 0.05 | 1 − 0.05/0.95 = 0.947 |
| 8% crashes (medium) | 0.08 | 1 − 0.08/0.92 = 0.913 ≈ P90 |

| Situation | p | q* |
|---|---|---|
| 2% crashes (low) | 0.02 | 0.980 |
| 100 successes, 0 failures (computed) | 0.0083 | 0.992 |
| …then 8 failures (computed) | 0.0703 | 0.924 |

**Result:** at 8% crashes P90 is already near-optimal, which explains why heavy-tailed results look "bad" at that rate: it is the price of fast recovery. With no failures, duplicates on heavy-tailed jobs fell from 9.4% to 4.0% (H6, README). **Assumption:** the controller can observe crashes (SQS alone only shows redeliveries). The cost ratio needs calibration: r = 0.25 measured better than r = 1 (README, Phase 4).

## 7.7 Page–Hinkley change detection

**Intuition:** keep a running total of how far each new value is above the running mean. If values suddenly rise, the total climbs steadily; normal noise makes it wander up and down. Raise an alarm when it has climbed more than λ above its lowest point (and symmetrically for falls).

```
x_t = log(duration_t);   mean_t = mean_{t-1} + (x_t - mean_{t-1}) / t
U_t = U_{t-1} + (x_t - mean_t - delta);   alarm if U_t - min(U) > lambda      (rise)
L_t = L_{t-1} + (x_t - mean_t + delta);   alarm if max(L) - L_t > lambda      (fall)
delta = 0.05,  lambda = 3  (x max(1, std(log history) / 0.25) when spread-scaled)
```

**On an alarm:** keep only the last 3 durations of that class, so the window refills with post-change values instead of waiting for 40 completions to push the old regime out. **Example 5** in Chapter 9 traces a doubling detected on the 6th new sample. **Measured:** false alarms on Pareto jobs 3.65 per run unscaled vs 0.80 scaled (H17); duplicates after the shift 5.9% → 4.9% unscaled (H9) and 5.9% → 5.2% scaled (H18). **Assumption:** roughly log-normal noise.

## 7.8 Adaptive heartbeat formulas

```
lease     = min(lease_cap, P90(history) + margin)          lease_cap = 5 s
interval  = 0.7 x lease
extension = lease  (each heartbeat sets deadline = now + lease)
budget    = clamp(P99(history) x 1.25 + 2, lease, 40 s)    from processing start
deadline never exceeds start + budget
```

| Class (computed example) | P90 + 2 | Lease | Interval | P99 | Budget |
|---|---|---|---|---|---|
| short (1.1, 1.3, 0.9, 1.2, 1.4) | 3.36 | 3.36 | 2.35 | 1.40 | 3.75 |
| long (13.1 … 13.7, 7 samples) | 20.12 | 5.00 | 3.50 | 19.36 | 26.20 |
| any class, cold (no history) | — | 5.00 | 3.50 | — | 40 (global) |

**Why this shape:** short jobs finish inside their first lease and never heartbeat; long jobs are capped at 5 s so a crash is noticed within 5 s whatever the job length; a hung long job is reclaimed after about 26 s instead of 40 s. **Dynamic budget:** the budget is recomputed at every heartbeat, so a job already running when durations grow gets the larger budget at once.

## 7.9 Workload and failure randomness

```
short: lognormal(log 1.2, 0.22)   medium: lognormal(log 5, 0.28)   long: lognormal(log 14, 0.32)
heavy:  (pareto(2.2) + 1) x 3        shifted jobs: same with scale x 2
all clipped to [0.15, 90] s;  gaps ~ exponential(mean 1/1.2 s)
failure key = (seed, crc32(job_id), receive_count);  fraction ~ uniform(0.15, 0.85)
retry jitter: lognormal(-s^2/2, s), s = 0.2 (mean 1);  worker speed: same form, s = 0.25
```

The $-\sigma^2/2$ shift makes the log-normal factor's mean exactly 1, so jitter adds noise without making jobs slower on average (tested by `test_jitter_has_mean_one`).

## 7.10 Statistics formulas

| Formula | Meaning |
|---|---|
| Paired difference d_s = metric(reference, seed s) − metric(comparator, seed s) | Negative = reference lower. |
| Mean ± t(0.975, n−1) × SD/√n | Student-t 95% interval (20 seeds: t ≈ 2.09). |
| Bootstrap: resample seeds with replacement 2,000 times; 2.5% and 97.5% of the means | Interval without a normality assumption. |
| W+ = sum of ranks of \|d\| where d > 0 | Wilcoxon statistic; exact p by enumerating all 2^n sign patterns with DP. |
| Holm: p_(k) × (m − k + 1), running maximum | Family-wise error control. |
| A dominates B if A ≤ B on both axes and < on one | Pareto front = non-dominated policies. |

## 7.11 Closed-form results used to verify the simulator

| Check | Formula | Test |
|---|---|---|
| Receives per job, fixed timeout T, no contention | ceil(D / T) | test_receives_per_job_equal_ceil_duration_over_timeout |
| Share of jobs redelivered | P(D > T) (count of durations above T) | test_duplicate_share_matches_tail_probability |
| Crash recovery | T − f × D | test_crash_recovery_equals_timeout_minus_progress |
| Heartbeat count | jobs of 7.5 s, interval 1 s → 7 | test_heartbeat_extension_count |
| M/M/c queue mean wait (Erlang C) | W_q = P_wait / (cμ − λ) | test_mean_wait_matches_erlang_c (λ=3, μ=1, c=4, 30,000 jobs, within 10%) |

## 7.12 Computational complexity

- **Event loop:** each push/pop on the heap costs O(log E). Stale events are left in the heap and skipped, which is simpler than deleting them.
- **Dispatch:** sorts the visible list after every event, O(V log V). Fine for 200 jobs; it would be a bottleneck for very large queues.
- **Policy decision:** sorting at most 40 values per receive. Page–Hinkley is O(1) per observation (O(W) with spread scaling).
- **Wilcoxon exact:** the DP array has n(n+1)/2 entries updated n times: about $n^3/2$ operations, trivial for n = 20.
- **Pareto flags:** $O(k^2)$ for k policies per scenario.

- **Whole full run:** 7,060 simulations; the README says about 1,700 simulations for an earlier version of full.json. Runtime is not recorded in the repository.

CHAPTER 8

# Technologies, Tools and Dependencies

| Technology | What it is | Why and how the project uses it | Essential? |
|---|---|---|---|
| Python 3 | General-purpose language. | All code. `from __future__ import annotations` allows modern type hints on older 3.x. | Yes |
| NumPy | Numerical arrays and random generators. | Seeded `default_rng` (keyed for CRN), log-normal/Pareto/exponential draws, percentiles, sorting. | Yes |
| pandas | Tables (DataFrames). | Attempt and run tables, grouping for aggregates and pairing, CSV and HTML table output. | Yes |
| matplotlib | Plotting library. | All figures (Agg backend for files). | Optional; run continues without it |
| seaborn | Statistical plots on matplotlib. | Bar, line and scatter charts with SD error bars. | Optional |
| unittest / pytest | Test frameworks. | 110 tests; pytest runs both styles; `conftest.py` puts the project on sys.path. | For development |
| boto3 | AWS SDK for Python. | SQS client in the replication harness; uses the standard credential chain (never stored in code). | Only for real runs; imported lazily |
| Amazon SQS | Managed message queue. | The real system being modelled and the replication target (Standard queue `adaptive-visibility-test`). | Only for real runs |
| AWS IAM | Access control. | `docs/iam-policy.json` grants only the 5 calls on one queue. | For real runs |
| AWS CloudShell | Browser terminal already logged in to AWS. | How the real run was executed (no access keys needed). | Replaceable (any machine with credentials) |
| Google Colab | Hosted Jupyter notebooks. | `Adaptive_SQS_Controller_Colab.ipynb`: install, test, demo run, plots, optional full run and SQS check. | Optional |
| Git / GitHub | Version control. | 17 commits; hypotheses committed before evaluation runs (commit hashes cited in methodology.md). | For reproducibility claims |
| PowerShell | Windows shell. | `scripts/generate_plots_windows.ps1` draws basic plots with System.Drawing when matplotlib is unavailable. | Optional fallback |
| Standard library | Built-in modules. | `heapq` event queue; `dataclasses`; `collections.deque` windows; `uuid` attempt ids and receipts; `zlib.crc32` stable job keys; `threading` replication workers and ledger lock; `argparse`, `json`, `pathlib`, `webbrowser`. | Yes |

## 8.1 Installation and configuration

```
Commands

python -m pip install -r requirements.txt        # numpy, pandas, matplotlib, seaborn, ...
python -m pytest -q                              # run the tests
python -m src.experiment --config configs/smoke.json --output results/smoke --plots
python -m src.experiment --config configs/full.json  --output results/full  --plots
python -m src.replication --fake --time-scale 0.2                  # offline dry run
python -m src.replication --queue-url <URL> --region <REGION>        # real SQS
```

- Real runs need AWS credentials through the normal chain (environment, ~/.aws, an IAM role, or CloudShell). The README and docs repeatedly say never to put them in code, notebooks, CSVs or Git.
- In CloudShell, Python runs in a virtual environment, so `pip install --user` fails; plain `pip3 install` works.
- Why the statistics are hand-written: the stats module's docstring says "implemented without SciPy". The reason is not documented; the likely benefit is fewer dependencies and full control, at the cost of more code to test (it is tested).

CHAPTER 9

# Data Flow and Worked Examples

Every number in this chapter was produced by running the project's own code (a trace script calling `SQSSimulator`, the policies and the workload generator directly). Nothing is hand-calculated except where marked.

## 9.1 Example 1: three jobs with a 3-second timeout

**Initial state:** 4 idle workers, `fixed_short` (3 s), no failures. Jobs: job-0 short 1.2 s sent at 0.0; job-1 long 7.0 s sent at 0.5; job-2 medium 4.0 s sent at 1.0.

| Time | Event | Decision / state change |
|---|---|---|
| 0.0 | send job-0; dispatch | worker-00 receives job-0 (count 1); deadline 3.0; complete scheduled at 1.2 |
| 0.5 | send job-1; dispatch | worker-01 receives job-1; deadline 3.5; complete at 7.5 |
| 1.0 | send job-2; dispatch | worker-02 receives job-2; deadline 4.0; complete at 5.0 |
| 1.2 | complete job-0 | ledger commits job-0 → deleted; worker-00 free |
| 3.0 | expire job-0 | ignored (deleted) |
| 3.5 | expire job-1 | still current receipt → visible; worker-00 receives it (count 2, **duplicate processing**); deadline 6.5 |
| 4.0 | expire job-2 | visible; worker-03 receives it (count 2, duplicate); deadline 7.0 |
| 5.0 | complete job-2 #1 | commits job-2 → deleted |
| 6.5 | expire job-1 #2 | visible again; worker-02 receives (count 3, duplicate); deadline 9.5 |
| 7.0 | expire job-2 #2 | ignored (deleted) |
| 7.5 | complete job-1 #1 | commits job-1 → deleted |
| 8.0 | complete job-2 #2 | ledger rejects → duplicate committed result |
| 10.5 | complete job-1 #2 | rejected |
| 13.5 | complete job-1 #3 | rejected; all done, no busy workers → stop |

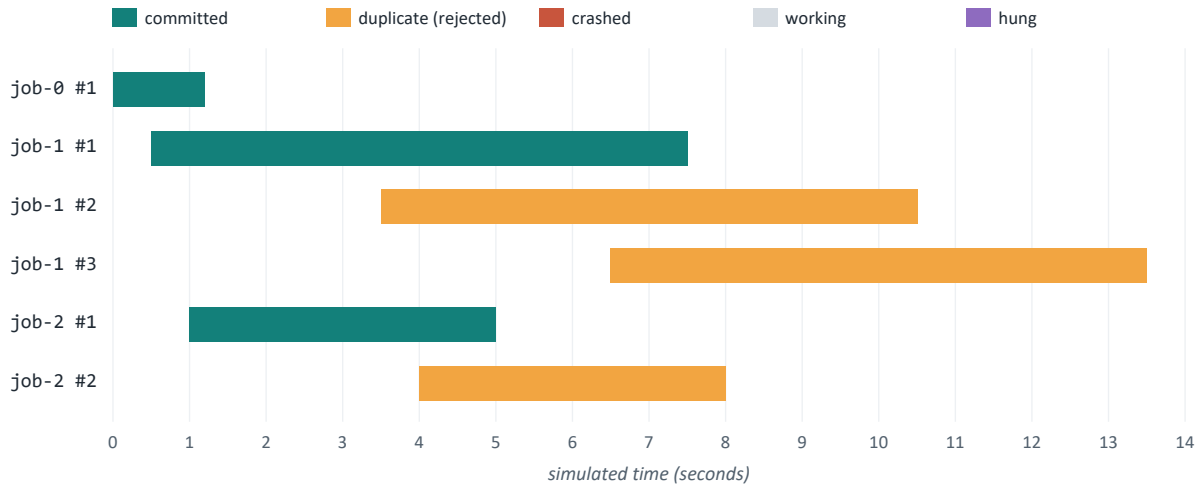



**Figure 5.** Example 1 as a timeline (traced by the simulator). Every attempt that started while another of the same job was running is a duplicate.

**Final output** (from `summarize_run`): 6 attempts; 3 flagged duplicate processing → duplicate rate **0.5**; mean latency (1.2 + 7.0 + 4.0)/3 = **4.07 s**; wasted time 4 + 7 + 7 = **18 s**; API calls 3 sends + 6 receives + 3 deletes = 12 → **4**

**per job**. **Why:** 3 s is shorter than two of the three jobs, so they reappear while still running (job-1, 7 s, appears ceil(7/3) = 3 times, matching the closed-form rule).

## 9.2 Example 2: the same jobs with a fixed heartbeat

`fixed_heartbeat` (lease 5 s, heartbeat 3.5 s, max 4 extensions, budget 40 s), same jobs:

| Job | Receive | Heartbeats | Deadline after heartbeat | Completes | Extensions |
|---|---|---|---|---|---|
| job-0 (1.2 s) | 0.0 | none (done at 1.2 < 3.5) | 5.0 | 1.2 | 0 |
| job-1 (7.0 s) | 0.5 | 4.0 | 9.0 | 7.5 | 1 |
| job-2 (4.0 s) | 1.0 | 4.5 | 9.5 | 5.0 | 1 |

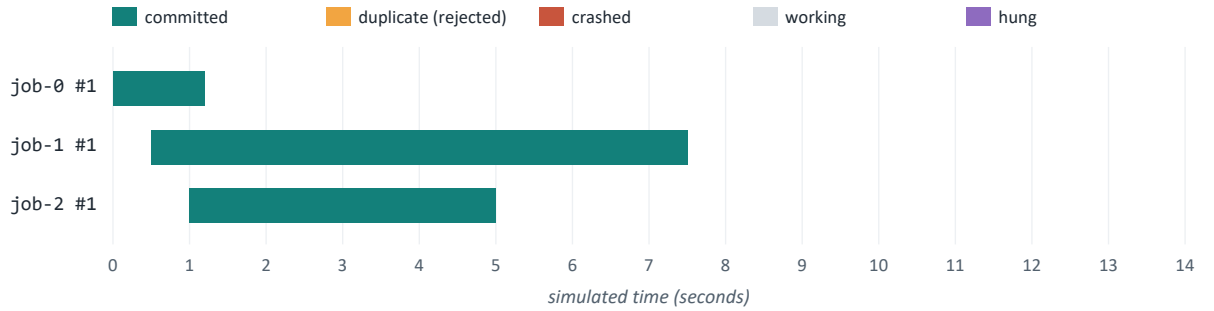



**Figure 6.** Example 2: no duplicates. Each long job is kept hidden by one heartbeat.

**Output:** 3 attempts, no duplicates, API calls 3 + 3 + 3 + 2 change-visibility = 11. **Trade-off:** 2 extra API calls instead of 3 duplicate attempts and 18 s of wasted work.

## 9.3 Example 3: the adaptive controller learning a class

`adaptive_p90` (margin 2 s, min_history 5, default 12 s, partial cold start). Committed durations of class *long* arrive one by one. Values are from the code:

| After sample | History n | Rule used | Prediction | Timeout | Fallback? |
|---|---|---|---|---|---|
| (none) | 0 | default | 12.00 | 12.00 | yes |
| 13.1 | 1 | 1.5 × max | 19.65 | 21.65 | yes |
| 15.8 | 2 | 1.5 × max | 23.70 | 25.70 | yes |
| 12.4 | 3 | 1.5 × max (max still 15.8) | 23.70 | 25.70 | yes |
| 17.2 | 4 | 1.5 × max | 25.80 | 27.80 | yes |
| 14.0 | 5 | P90 of 5 | 16.64 | 18.64 | no |
| 19.5 | 6 | P90 of 6 | 18.35 | 20.35 | no |
| 13.7 | 7 | P90 of 7 | 18.12 | 20.12 | no |

**What to notice:** during cold start the controller is deliberately generous (21–28 s) to avoid duplicating long jobs; once 5 samples exist it tightens to about P90 + 2 s, which shortens crash recovery. The old rule would have used 12 s for the first four long jobs, all of which take longer than 12 s.

## 9.4 Example 4: crash and hang recovery

One 8 s job on one worker; the first attempt fails at 25% (crash) or hangs at 50%. Traced by the simulator:

| Policy | Fault | Fault time | Message visible again | Recovery | Completes |
|---|---|---|---|---|---|
| Fixed 10 s timeout | crash at 2 s | 2.0 | 10.0 (lease end) | **8.0 s** | 18.0 |
| Fixed heartbeat (5 s lease) | crash at 2 s | 2.0 | 5.0 (first lease end; no heartbeat yet) | **3.0 s** | 13.0 |
| Fixed heartbeat | hang at 4 s | 4.0 | 19.0 (after 4 extensions: 8.5, 12, 15.5, 19) | **15.0 s** | 27.0 |

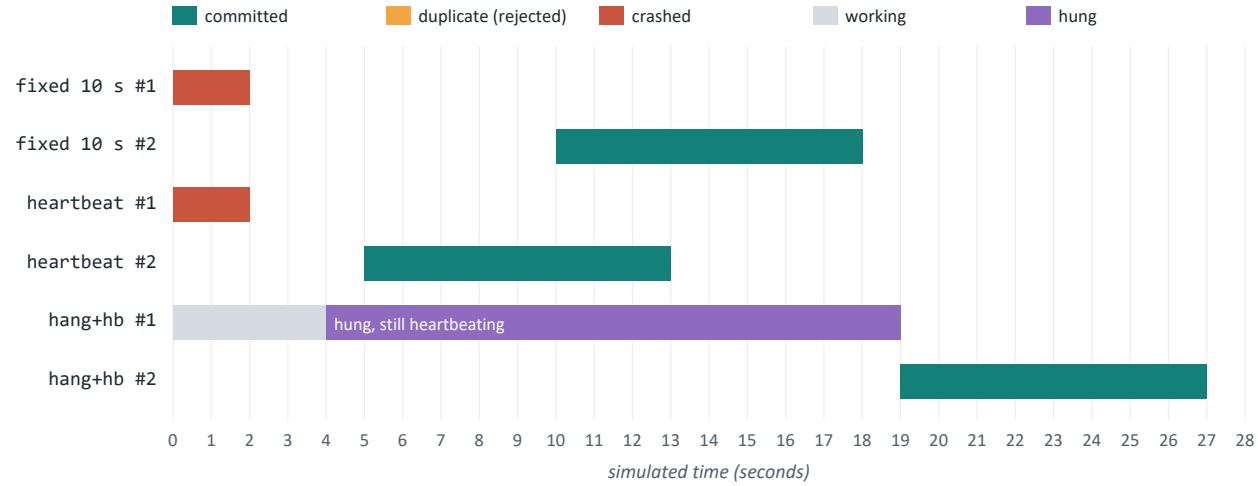



**Figure 7.** Example 4: a short lease recovers crashes quickly; a hang is only stopped by the extension limit or budget.

**Why:** crash recovery is driven by the length of the current lease; hang recovery by the total budget. This is exactly why the adaptive heartbeat keeps a short lease but learns a per-class budget.

## 9.5 Example 5: detecting a workload shift (Page–Hinkley)

A class receives 20 medium durations (median 5 s), then durations double (median 10 s). `adaptive_p90_cpd` (λ = 3, δ = 0.05). State = [n, mean, up, min_up, …]; window size shown last. From the code:

| Sample # | Duration | Running mean (log) | up | min_up | up − min_up | Window |
|---|---|---|---|---|---|---|
| 20 (last old) | 4.36 | 1.555 | −0.744 | −0.744 | 0.00 | 20 |
| 21 (first new) | 8.19 | 1.581 | −0.273 | −0.744 | 0.47 | 21 |
| 22 | 11.68 | 1.621 | 0.514 | −0.744 | 1.26 | 22 |
| 23 | 9.82 | 1.650 | 1.099 | −0.744 | 1.84 | 23 |
| 24 | 8.48 | 1.670 | 1.516 | −0.744 | 2.26 | 24 |
| 25 | 11.22 | 1.700 | 2.183 | −0.744 | 2.93 | 25 |
| 26 | 12.62 | — | — | — | > 3 → **alarm** | cut to 3 |

**Result:** the change is detected on the 6th new sample. The window then holds only 3 post-change durations, so the next P90 reflects the new regime almost at once, instead of after about 40 completions. This is what H9 measures.

## 9.6 Example 6: why the realistic baseline goes stale

For the mixed workload with seed 11, the per-class timeouts (P90 + 2 s) are (computed): oracle short 3.56, medium 9.21, long 21.75 s; pre-tuned short 3.35, medium 9.08, long 20.08 s. They are close, so in stationary workloads pre-tuning works well. In the sudden-shift workload (all medium), the pre-tuned timeout is **8.4 s**, fitted to the first 20% of a pilot run (median 5 s). After the shift at t ≈ 90.7 s the median becomes 10 s, so most jobs now exceed 8.4 s and are duplicated. Measured over 20 seeds: 20.6% duplicate processing for pre-tuned vs 5.9% for adaptive P90 (H1).

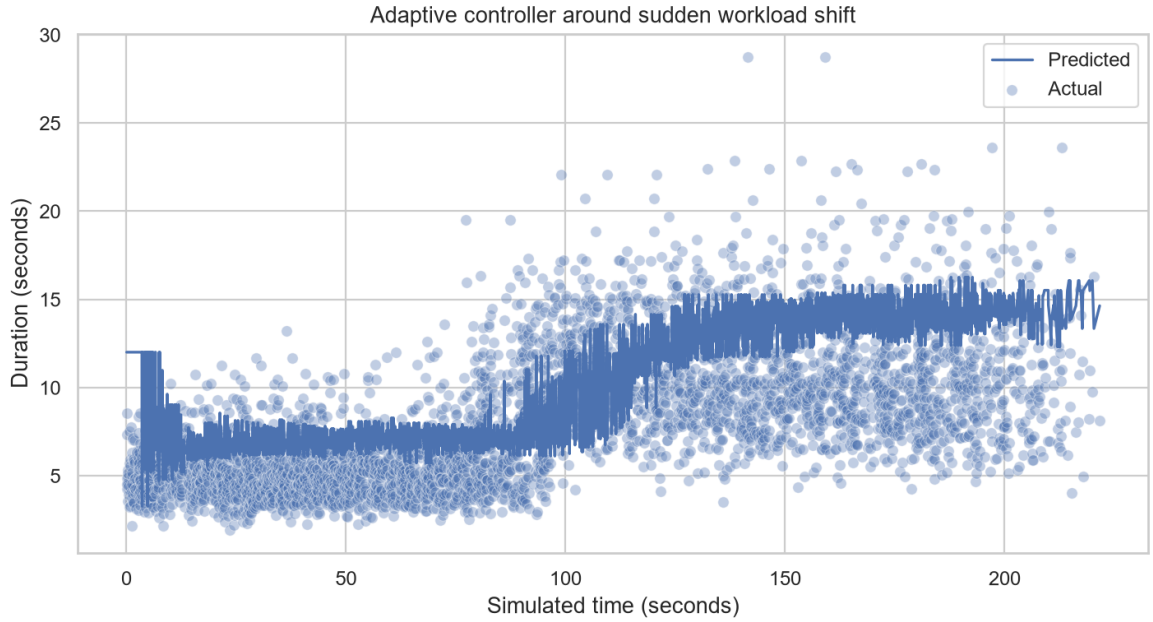



**Figure 8.** The adaptive P90 prediction (line) tracking actual durations (dots) around the sudden shift (results/full/plots/08_shift_behaviour.png, all 20 seeds overlaid). The prediction starts at the 12 s default, settles near 7 s, and climbs to about 14 s after the doubling.

CHAPTER 10

# Testing, Evaluation and Results

## 10.1 How the project is tested

```
Terminal

python -m pytest -q          # 110 passed (run on 22 Oct 2026)
```

| Test file | Tests | What it protects |
|---|---|---|
| `test_all.py` | 11 | Core behaviour in one place: policy calculations, fallback, idempotency, redelivery, deletion, failure, extension limit, single commit, seed reproducibility. |
| `test_policies.py`, `test_simulator.py`, `test_workload.py`, `test_idempotency.py` | 11 | The same basics in pytest style; the cold-start test checks both the partial and the fixed rule. |
| `test_phase1_fixes.py` | 7 | Dispatch bug fix, common random numbers, t critical values, t/bootstrap intervals, no prediction metrics for static policies. |
| `test_phase2.py` | 15 | Relative/MAD margins, pre-tuned baseline independence and staleness, warm-up metrics, Wilcoxon, Holm, Pareto, pairing. |
| `test_phase3.py` | 12 | Heartbeat until crash, hang until budget, hang draws independent of crash draws, adaptive lease and budget, ablation. |
| `test_phase4.py` | 17 | Partial cold start, conformal rank and coverage, cost-aware quantile, liveness probe, coverage metric, primary-test logic. |
| `test_phase4_3.py` | 9 | Page–Hinkley detects rises and falls, rarely alarms on stationary noise, dynamic budget reaches in-flight jobs. |
| `test_phase5.py` | 15 | DLQ, poison messages, jitter and speeds (seeded, mean one), delete and heartbeat latency, batching, outages, spread scaling, cost budget. |
| `test_verification.py` | 7 | Simulator against closed-form results and Erlang C (Section 7.11). |
| `test_replication.py` | 6 | Fake queue rules, integer rounding, refusing a non-empty queue, releasing foreign messages, fake run vs simulator, report page. |

## 10.2 Important test cases by type

| Type | Example tests | What they show |
|---|---|---|
| Normal | test_fake_run_completes_and_compares_with_simulator; test_run_writes_new_policies_and_paired_table | End-to-end runs complete and write the expected tables. |
| Boundary | test_heartbeat_extensions_stop_at_cap (redelivery at exactly 5.0 s); test_too_few_samples_falls_back_to_cold_start; test_zero_warmup_matches_full_metrics; test_quantile_clamped | Limits behave exactly as designed. |
| Invalid input | test_negative_margin_rejected; test_invalid_parameters_rejected (3 files); test_poison_without_dlq_rejected; test_refuses_non_empty_queue | Bad settings fail early and loudly. |

| Type | Example tests | What they show |
|---|---|---|
| Failure scenarios | test_worker_failure_causes_later_recovery; test_hung_worker_heartbeats_until_budget_then_is_reclaimed; test_outage_crashes_running_attempts_on_its_workers; test_late_heartbeat_loses_the_lease; test_delete_latency_can_cause_a_duplicate | Crashes, hangs, outages and latency races are handled correctly. |
| Fairness / reproducibility | test_policies_see_identical_failures_for_same_attempt; test_outages_are_seeded_and_policy_independent; test_poison_marking_leaves_workload_unchanged; test_reproducibility_with_fixed_seed | Comparisons are paired and repeatable. |
| Correctness against theory | test_mean_wait_matches_erlang_c; test_duplicate_share_matches_tail_probability | The simulator agrees with independent maths. |

## 10.3 Evaluation design

| Element | Choice |
|---|---|
| Independent variables | Policy, workload, crash rate, hang rate, margin, window, percentile, min_history, budget factor, cost ratio, DLQ limit, realism options, seed. |
| Dependent variables | Duplicates, wasted work, latency, throughput, recovery (crash and hang), prediction error and coverage, API calls, adaptation time, DLQ metrics, trade-off cost. |
| Main run | configs/full.json: 200 jobs, 10 workers, 1.2 jobs/s, 20 seeds, 28 scenarios, 31 policies → 7,060 simulations. |
| Fairness | Same jobs for all policies; common random numbers; realistic and oracle static baselines; heartbeat policies share the 5 s lease and 40 s cap. |
| Tuning vs evaluation | Design choices tuned on seeds 1001–1010; evaluation on 11–220. |
| Analysis order | Primary hypotheses first; then pre-tuned, oracle and global baselines; effect sizes in original units; full and steady-state values; simulator and AWS evidence kept separate (methodology.md). |

## 10.4 Measured results: main scenarios

20-seed means from `results/full/aggregate_metrics.csv` (margin 2 s, window 40). Dup = duplicate-processing rate; Rec = mean failure recovery; Lat = mean end-to-end latency; Cost = trade-off cost in seconds per job.

| Policy | Dup | Dup after warm-up | Rec (s) | Lat (s) | API/job | Cost |
|---|---|---|---|---|---|---|
| fixed_short | 44.2% | 44.0% | 2.50 | 71.6 | 3.91 | 10.98 |
| fixed_long | 0.1% | 0.2% | 27.46 | 7.8 | 3.09 | 2.38 |
| fixed_overall_mean | 30.6% | 30.4% | 4.12 | 33.4 | 3.55 | 6.89 |
| fixed_per_class (oracle) | 1.4% | 1.4% | 6.46 | 6.06 | 3.10 | 0.86 |
| pretuned_per_class | 2.4% | 2.3% | 6.34 | 6.15 | 3.11 | 1.03 |
| adaptive_p90 | 3.1% | 1.8% | 6.37 | 6.24 | 3.12 | 1.17 |
| adaptive_conformal | 2.5% | 1.2% | 7.14 | 6.21 | 3.11 | 1.10 |
| fixed_heartbeat | 2.7% | 2.7% | 3.88 | 5.98 | 4.12 | 0.98 |
| adaptive_heartbeat | 0.2% | 0.3% | 3.18 | 5.75 | 4.15 | 0.33 |
| adaptive_heartbeat_dynamic | 0.2% | 0.2% | 3.17 | 5.70 | 4.15 | 0.31 |

Scenario mixed_with_failures (mixed workload, 8% crashes).

| Policy | Dup | Rec (s) | Lat (s) | Cost |
|---|---|---|---|---|
| fixed_per_class (oracle) | 2.7% | 11.60 | 13.65 | 0.66 |
| pretuned_per_class | 20.6% | 5.90 | 25.69 | 3.39 |
| adaptive_p90 | 5.9% | 8.53 | 16.52 | 0.99 |
| adaptive_p90_cpd_scaled | 5.2% | 8.58 | 15.75 | 0.90 |
| adaptive_cost_aware | 3.4% | 9.86 | 14.82 | 0.66 |
| fixed_heartbeat | 0.3% | 3.51 | 12.18 | 0.13 |
| adaptive_heartbeat_dynamic | 0.4% | 3.58 | 12.34 | 0.13 |

Scenario sudden_shift (durations double halfway, 2% crashes).

| Policy | Dup | Crash+hang rec (s) | Hang rec (s) | Lat (s) | Cost |
|---|---|---|---|---|---|
| adaptive_p90 | 2.9% | 6.19 | 6.12 | 6.76 | 1.23 |
| fixed_heartbeat | 2.3% | 14.13 | 16.33 | 8.93 | 2.07 |
| adaptive_heartbeat | 0.2% | 9.85 | 11.34 | 7.01 | 1.07 |
| adaptive_heartbeat_dynamic | 0.2% | 8.76 | 10.05 | 6.74 | 0.95 |

Scenario mixed_with_hangs (2% crashes, 8% hangs).

| Policy | Dup | Hang rec (s) | Lat (s) | Cost |
|---|---|---|---|---|
| fixed_heartbeat | 2.1% | 16.46 | 12.52 | 2.50 |
| adaptive_heartbeat | 2.2% | 20.80 | 13.74 | 2.68 |
| adaptive_heartbeat_dynamic | 2.2% | 20.30 | 12.97 | 2.67 |
| adaptive_p90 | 8.3% | 7.87 | 18.99 | 3.40 |

Scenario heavy_tailed_with_hangs: the one place the fixed heartbeat still wins.

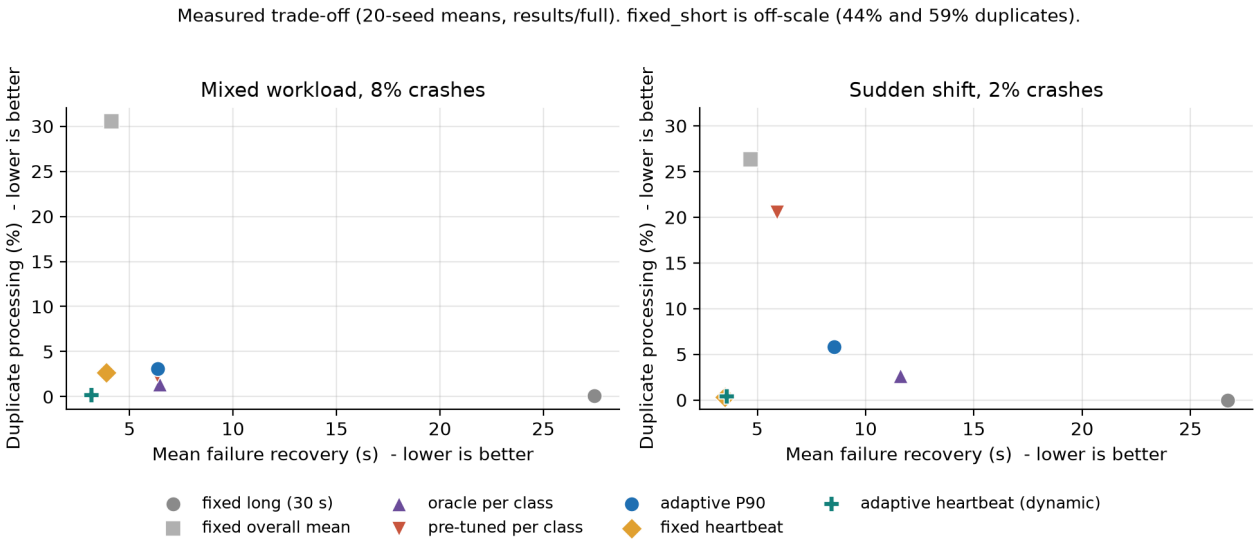



**Figure 9.** Measured duplicate/recovery trade-off for seven policies in two scenarios (made for this guide from results/full/aggregate_metrics.csv). Bottom-left is best. In the left panel the pre-tuned point is hidden behind the adaptive P90 and oracle points (6.3 s, 2.4%).

## 10.5 Primary hypotheses: 18 of 18 supported

| ID | Claim (reference vs comparator, scenario) | Mean diff. | Seeds in direction | Holm p |
|---|---|---|---|---|
| H1 | Adaptive P90 fewer duplicates than pre-tuned (shift) | −14.7 pts | 20/20 | <0.0001 |
| H2 | Adaptive heartbeat faster recovery than fixed heartbeat (mixed + hangs) | −4.28 s | 20/20 | <0.0001 |
| H3 | Adaptive heartbeat fewer duplicates (mixed + hangs) | −2.2 pts | 20/20 | <0.0001 |
| H4 | Adaptive heartbeat fewer duplicates (mixed + crashes) | −2.5 pts | 20/20 | 0.0008 |
| H5 | Partial cold start fewer duplicates than fixed fallback (mixed, no failure) | −0.95 pts | 13/20 (7 ties) | 0.0049 |
| H6 | Cost-aware fewer duplicates than P90 (heavy-tailed, no failure) | −5.4 pts | 20/20 | 0.0008 |
| H7 | Cost-aware lower trade-off cost than P90 (shift) | −0.33 s/job | 20/20 | <0.0001 |
| H8 | Conformal fewer duplicates than P90 (mixed + crashes) | −0.63 pts | 13/20 (7 ties) | 0.0012 |
| H9 | Change detection lowers P90 duplicates (shift) | −0.97 pts | 17/20 | 0.0002 |
| H10 | Dynamic budget lowers heartbeat duplicates (shift) | −0.49 pts | 12/20 | 0.0050 |
| H11 | Dynamic budget faster hang recovery (mixed + hangs) | −1.29 s | 15/20 | 0.0121 |
| H12 | Adaptive P90 fewer false DLQ than overall mean (maxReceiveCount 3) | −4.6 pts | 20/20 | 0.0008 |
| H13 | H1 survives realism (realistic shift) | −10.4 pts | 20/20 | <0.0001 |
| H14 | Dynamic heartbeat faster hang recovery than fixed (realistic hangs) | −5.32 s | 20/20 | <0.0001 |
| H15 | Dynamic heartbeat fewer duplicates than fixed (realistic hangs) | −2.6 pts | 20/20 | <0.0001 |
| H16 | Dynamic heartbeat quarantines poison faster (realistic poison) | −3.60 s | 20/20 | <0.0001 |
| H17 | Scaled detection fewer false alarms (heavy-tailed) | −2.85 per run | 19/20 | 0.0008 |
| H18 | Scaled detection still lowers P90 duplicates (shift) | −0.62 pts | 13/20 | 0.0012 |

Source: results/full/primary_hypotheses.csv. pts = percentage points.

> **Caution**
>
> H1–H4 are labelled **post hoc** in the configuration: they re-test findings from earlier phases on the same seeds. H5–H18 were committed before the evaluation run that tests them. H10, H11, H18 have smaller effects with some tied or reversed seeds; they are supported statistically but the practical gains are modest (under 1.5 s or 1 point).

## 10.6 Real Amazon SQS replication

Run in AWS CloudShell against a Standard queue `adaptive-visibility-test`: 40 mixed jobs, 4 worker threads, seed 11, 8% crashes, time scale 1.0, one run per policy (`results/aws_replication/replication_summary.csv`).

| Policy | Metric | Real SQS | Simulated twin | Difference |
|---|---|---|---|---|
| fixed_short | Duplicate processing | 20.0% (10 of 50) | 28.6% (16 of 56) | −8.6 pts |
| | Attempts | 50 | 56 | −6 |
| | Mean recovery (3 crashes) | 7.96 s | 2.28 s | +5.67 s |
| | Mean latency | 27.0 s | 44.7 s | −17.6 s |

| Policy | Metric | Real SQS | Simulated twin | Difference |
|---|---|---|---|---|
| fixed_heartbeat | Duplicates / attempts / extensions | 0% / 41 / 28 | 0% / 41 / 28 | exact |
| | Recovery (1 crash) | 3.55 s | 4.12 s | −0.57 s |
| | Latency / makespan | 24.7 / 53.4 s | 20.3 / 44.6 s | +4.4 / +8.8 s |
| adaptive_p90 | Duplicate processing | 2.4% (1 of 42) | 4.7% (2 of 43) | −2.3 pts |
| | Recovery (1 crash) | 5.49 s | 19.46 s | −13.97 s |
| | Latency | 24.95 s | 24.53 s | +0.42 s |
| adaptive_heartbeat_dynamic | Duplicates / attempts / extensions | 0% / 41 / 28 | 0% / 41 / 28 | exact |
| | Recovery (1 crash) | 9.77 s | 4.12 s | +5.65 s |
| | Latency / makespan | 23.8 / 46.3 s | 20.3 / 44.6 s | +3.5 / +1.7 s |

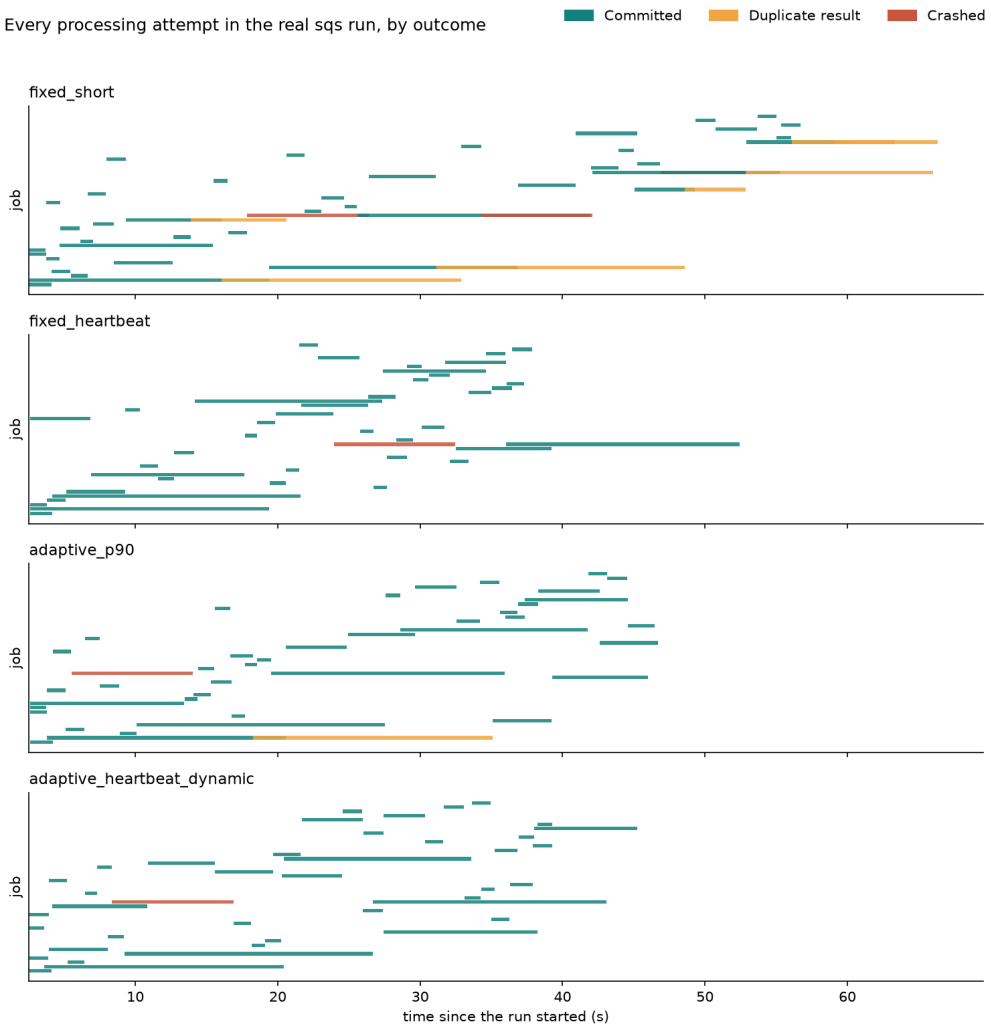



**Figure 10.** Every processing attempt in the real SQS run, by outcome (results/aws_replication/plots/07_attempt_timeline.png). Orange bars are duplicate attempts rejected by the ledger; red bars are injected crashes.

**What this shows:**

- **Mechanics agree.** Attempt and heartbeat-extension counts match almost exactly; the duplicate ranking (fixed_short > adaptive_p90 > heartbeat policies = 0) is preserved.
- **Real SQS produced fewer duplicates than predicted.** Plausible reasons (not separately measured): real API calls and polling take time the simulator does not charge; the harness sets the policy lease just after receiving,

so the effective lease starts slightly later; SQS does not re-show messages at the exact millisecond.

- **Heartbeat runs were slower in reality** (+3.5 to +4.4 s latency): each heartbeat and receive is a real network round trip.
- **Recovery comparisons are noise.** Three of the four policies had exactly one crash. The two heartbeat policies behave identically in simulation (4.12 s) yet recovered in 3.55 s and 9.77 s on SQS.
- **Idempotency mattered on real SQS:** fixed_short finished 7 jobs a second time; the ledger rejected those results.

## 10.7 What the results demonstrate

- Under a workload shift, a learned per-class timeout clearly beats a realistic static per-class configuration (H1, H13).
- In stationary workloads, adaptive P90's extra duplicates are mostly cold start: after warm-up it matches pre-tuned (1.8% vs 2.3% in mixed with failures).
- A short lease with heartbeats and a learned per-class budget gives the lowest trade-off cost in most scenarios, and beats a fixed heartbeat when workers can hang (H2, H3, H14, H15).
- Choosing the percentile from the failure rate explains when P90 is right ($q^* \approx 0.91$ at 8% crashes).
- Too-short timeouts wrongly dead-letter healthy jobs; adaptive and heartbeat policies avoid this (H12).
- The simulator's mechanics agree with closed-form results, queueing theory and one real SQS run.

## 10.8 What the results do not prove

- They do not show production-scale performance: the evidence is simulation of synthetic workloads; the real run is 40 jobs, once.
- They do not show that adaptation always wins: the oracle static policy has fewer duplicates in stationary and shifted workloads; the fixed heartbeat wins on heavy-tailed hangs.
- They do not validate the simulator's timing or recovery numbers on AWS (too few crashes in one run).
- They do not compare against published methods; there is no literature comparison in the repository.
- Real job-duration traces were not used, so generalisation to real workloads is untested.

## 10.9 Known issues: fixed and remaining

A review on 22 Oct 2026 found the issues below. The fixed ones were corrected in the repository the same day (not yet committed). None of the fixes touched simulator or policy code, so every result in this guide is unchanged.

| Issue | What was wrong | Status |
|---|---|---|
| Outdated test | `test_policies.py::test_insufficient_history_fallback` expected the fixed default timeout (13 s in the test) after one sample of 2 s, but the partial cold start (the default since Phase 4) returns 1.5 × 2 + 2 = 5 s. Stale compiled caches from an old copy of the project also made pytest show a wrong file path. | **Fixed:** the test now checks both rules; caches cleared; 110/110 pass. |
| README result tables | Phase 2 and Phase 3 tables predated later fixes (e.g. adaptive P90 4.0% vs 3.1% now); the smoke table described an old 36-run smoke. | **Fixed:** new "Current headline results" table from results/full; smoke table updated; phase tables labelled as historical. |
| README limitations | Said the simulator had zero latency and per-message calls, though Phase 5 added both. | **Fixed.** |

| Issue | What was wrong | Status |
|---|---|---|
| README Phase 6.1 | Said "not yet run against AWS". | **Fixed:** real-run table and interpretation added. |
| results/aws_replication | Held the offline fake run. | **Fixed:** replaced with the real AWS run (`"mode": "aws"`). |
| README controller text and docs/architecture.md | Described the old fixed cold-start fallback. | **Fixed:** now describe the partial cold start. |
| PyYAML | Listed in requirements.txt and the notebook but never imported. | **Fixed:** removed. |
| README ending | Stray `# adaptive-sqs-controller` heading. | **Fixed:** removed. |
| AWS guide | Did not describe the CloudShell route or the pip --user problem. | **Fixed:** CloudShell section added to docs/aws_replication.md. |
| Run time | No record of how long the full experiment takes. | Remaining (unverified). |
| Literature review | No related-work comparison in the repository. | Remaining (see Chapter 11). |
| Real-SQS evidence | One run per policy; too few crashes to judge recovery. | Remaining (needs repeated runs). |

CHAPTER 11

# Novelty and Design Decisions

## 11.1 What is distinctive about this project

- **A per-class heartbeat budget learned from the tail (P99 × 1.25 + 2 s), re-read at every heartbeat.** It separates the two jobs of a timeout: the short lease detects crashes; the learned budget bounds hangs per class. This combination, with measured gains against a fixed heartbeat when workers hang, is the most distinctive technical result.
- **An evaluation framework for visibility-timeout policies:** a verified SQS event model including hangs, DLQ, latency, outages and poison messages; common random numbers; realistic and oracle baselines; pre-declared hypotheses; and a real-SQS replication harness.
- **Honest negative results:** where adaptation loses (cold start, heavy tails, against the oracle and, for heavy-tailed hangs, against the fixed heartbeat) is reported, not hidden.

## 11.2 What already exists (and should be cited in a write-up)

The repository has no literature review. The following well-known ideas are close relatives. They are listed here from general knowledge so you can prepare; they are *not* cited anywhere in the project.

| Existing idea | How it relates | How this project differs |
|---|---|---|
| TCP retransmission timeout (Jacobson/Karels, 1988; RFC 6298) | Sets a timeout from a smoothed mean and variance of measured round-trip times. | Uses per-class percentiles of job durations, adds heartbeats, budgets and hang handling, and evaluates on a queue. |
| Leases (Gray and Cheriton, 1989) | Time-limited ownership that must be renewed. | The visibility timeout is a lease; the project learns its length and renewal limit. |
| Heartbeats and failure detectors | Detect dead processes by missing heartbeats. | Here a live-but-hung worker still heartbeats; the budget handles that case. |
| Speculative execution (MapReduce, Hadoop, Spark) | Re-run slow tasks on another machine. | Duplicates are the cost to avoid here, not a deliberate strategy. |
| AWS's own guidance on extending visibility with ChangeMessageVisibility | Recommends heartbeats for long jobs. | Project quantifies when heartbeats help and adds a learned per-class budget. |
| Conformal prediction; Page–Hinkley test; cost-based quantile choice | Standard statistical tools. | Applied to timeout setting; the project's contribution is the application and evaluation, not the tools. |

> **Caution**
>
> Combining existing tools is not by itself novel. In a viva, present the contribution as "an interpretable, per-class, heartbeat-aware timeout controller and a rigorous, reproducible evaluation of when it helps", not as a new algorithm.

## 11.3 Design decisions and the reasons for them

| Decision | Reason (from code comments or docs) | Trade-off |
|---|---|---|
| Percentile + margin, not machine learning | Interpretable, cheap, explainable; ML only if it shows added value (README). | Ignores job features beyond class. |
| Per class, not global | One number cannot fit 1 s and 15 s jobs. | Rare classes learn slowly. |
| Learn only from committed successes | Duplicates would weight a job twice (architecture.md). | Crashed attempts give no duration information. |
| Rolling window of 40 | Follows change; sweep showed 10–25 react faster to shifts. | Small windows are noisy. |
| Partial cold start, factor 1.5 | 1/(n+1) exceedance argument; factor tuned on tuning seeds. | Very generous timeouts at first. |
| Heartbeat lease capped at 5 s | Crash detected within one lease, whatever the job length. | ≈1 extra API call per job. |
| Budget = P99 × 1.25 + 2 s | Reclaim a hung job after about its class's worst normal duration. | Heavy tails give long budgets (the heavy-tailed hang gap). |
| Dynamic budget | In-flight jobs get a larger budget as soon as durations grow. | Worker must query the controller each heartbeat. |
| First success deletes | Keeps finite runs finite; declared simplification. | Differs slightly from real stale-handle behaviour. |
| Common random numbers | Paired comparisons; differences come from policies, not luck. | Failure key ignores scenario, so scenarios with the same seed share draws (limitations.md). |
| Wilcoxon + Holm, primary family | No normality assumption; control false positives; avoid cherry-picking. | Conservative; small effects may not reach significance. |
| Oracle and pre-tuned baselines | Beating a strong baseline is more informative (README viva notes). | Oracle is unrealistically strong by design. |
| Separate tuning seeds | Avoid tuning on the evaluation data. | Fewer seeds for tuning. |
| Integer timeouts in the replication | SQS only accepts whole seconds (1–43,200). | Real and simulated runs both round up. |
| No credentials in code; harness never creates queues | Safety and cost control (docs). | User must set up the queue and IAM manually. |

## 11.4 Implemented versus future work

| Implemented and evaluated | Implemented but exploratory or rejected | Future work (not implemented) |
|---|---|---|
| Fixed, per-class (oracle, pre-tuned), adaptive P90, partial cold start, conformal, cost-aware, Page–Hinkley (+ scaled), fixed and adaptive heartbeat, dynamic budget, DLQ, poison, jitter, worker speeds, API latency, batching, outages, real-SQS harness. | Relative margin (no better), MAD margin (unit-tested only), cost-conformal (degenerate when failures are rare), heartbeat + change detection (little gain), cost-aware heartbeat budget (rejected). | EWMA or streaming quantile sketches; explicit uncertainty; tuned loss function for margins; preregistered larger real-SQS replication; real traces; literature comparison; modelling autoscaling, in-flight quotas, retention expiry, SQS replicated-copy duplicates. |

CHAPTER 12

# Limitations and Future Improvements

## 12.1 Current limitations

| Area | Limitation | Source |
|---|---|---|
| Construct | Duplicate processing = overlap at attempt start; other definitions would give other rates. API counts are logical calls, not billing (long polls are billed even when empty). | limitations.md |
| Simulator fidelity | No autoscaling delay, in-flight quotas, retention expiry, or SQS's rare replicated-copy duplicates. Latency is a constant, not a distribution. Worker replacement is instant. | limitations.md |
| Deletion | First successful attempt deletes even with an older handle. | architecture.md |
| Hang model | Hung workers heartbeat until the budget and restart exactly at lease expiry; real supervisors differ. | limitations.md |
| Failure independence | Crash and hang rates are independent per attempt (outages are the only correlation). The failure key ignores the scenario. | limitations.md |
| Workloads | Synthetic distributions; no real traces. | assessment |
| Statistics | 20 seeds give moderate power; t intervals assume approximate normality of seed means; bootstrap is crude with few seeds. | limitations.md |
| Cost-aware rule | Assumes the controller sees worker health; derivation charges only overrun while the metric charges the full duplicate, so r needs calibration. | limitations.md |
| Conformal | Coverage holds only while durations are exchangeable; fails after shifts; degenerate above 40/41. | limitations.md |
| Change detection | Assumes roughly log-normal noise; false alarms on Pareto jobs (reduced by scaling). | limitations.md |
| Real validation | One 40-job run per policy; recovery based on 1–3 crashes; crash simulated inside a live thread. | replication data |
| Scalability | Dispatch sorts the visible list after every event; fine for hundreds of jobs, not for millions. | code reading |
| Security / reliability | In-memory ledger only (not durable); real deployments need a database or conditional writes. Credentials handled by AWS chain only. | code reading |
| Resources | No dedicated cost model; heartbeat policies use about 1 extra call per job. | results |

## 12.2 Suggested improvements and how each would help

| Improvement | How it helps |
|---|---|
| Write a related-work section (TCP RTO, leases, failure detectors, speculative execution, conformal, change detection). | Answers the first reviewer question: what is new? |
| Repeat the AWS replication 3–5 times per policy, with more jobs. | Turns a mechanics check into evidence about recovery and latency. |
| Use a public job-duration trace (e.g. Google or Alibaba cluster traces). | Tests generalisation beyond synthetic distributions. |
| Model latency as a distribution and add SQS replicated-copy duplicates. | Closes known gaps between simulator and reality (real duplicates were lower than simulated). |
| Durable idempotency (database with a unique key or conditional write). | Makes the controller deployable. |

| Improvement | How it helps |
| --- | --- |
| Streaming quantile sketches (t-digest, $P^2$) or EWMA. | Lower memory and faster updates for large windows. |
| Calibrate the cost ratio from real costs and choose q and margin from an explicit loss. | Principled tuning instead of fixed P90 + 2 s. |
| Add job features (input size) to predictions. | Better estimates within a class; compare with ML only if it helps. |

> **Note**
>
> Everything in the improvements table is a **suggestion**. None of it exists in the repository today.

CHAPTER 13

# Important Questions and Answers

Forty-one likely viva, presentation and interview questions, with short answers you can say out loud. Numbers come from results/full, the real AWS run, or the README, as cited in earlier chapters.

## 13.1 Basic concepts

**Q1. What is a visibility timeout?**

When a worker receives a message, SQS hides it for this period. If the worker does not delete it in time, the message becomes visible again so another worker can retry it. It is how SQS recovers from workers that die.

**Q2. Why can SQS deliver the same message twice?**

Standard queues are at-least-once: a message reappears when its timeout ends even if the first worker is still working, and SQS can also occasionally produce a duplicate copy on its own. The project models the first cause, not the second.

**Q3. What is the difference between duplicate delivery and duplicate processing?**

Duplicate delivery is any receive after the first. Duplicate processing means two attempts of the same job were running at the same time. A retry after a crash is a duplicate delivery but not duplicate processing. The headline metric is duplicate processing.

**Q4. If duplicates are rejected by the ledger, why do they matter?**

Idempotency protects the result, not the cost. Each duplicate still uses a worker for the full job, delays other jobs, and in systems without idempotency could cause double side effects.

**Q5. What is the trade-off the project studies?**

A short timeout recovers crashed jobs quickly but duplicates long jobs; a long timeout avoids duplicates but leaves crashed jobs hidden. The project measures both and looks for policies that improve both at once.

**Q6. What is a heartbeat and why is it useful?**

A periodic ChangeMessageVisibility call from a busy worker that pushes the deadline forward. It lets a short lease (fast crash detection) cover long jobs (no duplicates), at the cost of about one extra API call per job.

**Q7. What is the difference between a crash and a hang?**

A crash stops the worker and its heartbeats, so the message reappears after the current lease. A hang stops progress but the worker keeps heartbeating, so only a budget, an extension limit or a liveness probe ends it.

## 13.2 Architecture

**Q8. What are the main components?**

Workload generator, failure model, policies, the discrete-event simulator with its ledger, metrics, statistics, the experiment runner, and the replication harness for real SQS.

**Q9. How do the simulator and the policies communicate?**

Through the VisibilityPolicy interface: the simulator calls decide(), heartbeat_interval_for(), max_total_visibility_for(), extension_timeout(), observe() and observe_attempt_end(). Policies never call the simulator, so the same policy objects work on real SQS.

**Q10. Why a discrete-event simulator instead of real time?**

It is deterministic and fast: 7,060 runs of 200 jobs would take days in real time. Events are processed in time order and the clock jumps between them.

**Q11. How is the real-SQS run kept safe and cheap?**

It never creates or deletes queues, refuses a non-empty queue, only touches messages with its run id, deletes its leftovers, uses an IAM policy limited to five calls on one queue, and makes about 2,000–3,000 requests (inside the free tier).

**Q12. Why is the workload generated once per scenario and seed?**

So every policy sees exactly the same jobs; together with common random numbers for failures this makes comparisons paired.

## 13.3 Algorithms and implementation

**Q13. How does the adaptive controller compute a timeout?**

For the job's class it takes the last 40 committed durations, computes the 90th percentile and adds 2 s, clamped to 1–120 s. With fewer than 5 samples it uses 1.5 × the largest sample + 2 s, or 12 s if there are none.

**Q14. Why P90 and not the mean or the maximum?**

The mean is exceeded by many jobs (duplicates); the maximum makes crash recovery slow and is unstable. P90 covers most jobs and is interpretable. The sweep from P75 to P99 shows the expected smooth trade-off, and the cost-aware analysis shows $q^* \approx 0.91$ at 8% crashes.

**Q15. Why learn only from committed successes?**

A duplicate attempt of the same job would add its duration a second time and bias the estimate. Crashed attempts do not reveal the full duration.

**Q16. What does the partial cold start fix?**

The old 12 s fallback was shorter than long jobs (median 14 s), so early long jobs always duplicated. Using 1.5 × the largest sample adapts immediately. H5: about 1 point fewer duplicates, Holm $p = 0.005$.

**Q17. How does the adaptive heartbeat set its budget?**

Budget = P99 of the class × 1.25 + 2 s, between one lease and 40 s. A healthy job rarely exceeds it, while a hung worker is reclaimed after about the class's worst normal duration. The dynamic version recomputes it at every heartbeat.

**Q18. What is Page–Hinkley and why is it used?**

A sequential test that accumulates deviations of log-durations from their running mean; when the accumulated rise or fall exceeds a threshold it signals a change. The window is then cut to the last 3 values so the controller relearns quickly after a shift.

**Q19. What is the cost-aware quantile?**

$q^* = 1 - p \cdot r/(1 - p)$: the percentile that minimises expected overrun plus recovery cost given crash rate p and cost ratio r. p is learned with a Beta(1, 19) prior. Rare failures push $q^*$ up; frequent failures push it down.

**Q20. What does conformal prediction add?**

Choosing the ceil((n+1)q)-th order statistic guarantees at least q coverage for exchangeable data. Measured coverage rose from about 87% to 91% in the mixed scenario.

## 13.4 Code decisions

**Q21. Why is Message a frozen dataclass?**

It is shared by many policies and runs; making it immutable prevents accidental changes. Attempts, which do change, are a separate mutable class.

**Q22. How are stale events handled without deleting them from the heap?**

Each expiry event checks that the attempt's current deadline equals the event time and that its receipt is still current; otherwise it is ignored. Removing items from a heap would be more complex.

**Q23. Why does the simulator use seed + 1000 and the pilot seed + 500,000?**

To get independent random streams from the same experiment seed: the failure draws, and a pilot workload that never overlaps the evaluated jobs.

**Q24. Why are statistics implemented without SciPy?**

The module says so but does not give a reason; the effect is fewer dependencies. The functions are tested against reference values.

**Q25. Why does the replication wrap policies in IntegerTimeouts?**

SQS accepts only whole seconds between 1 and 43,200. Both the real run and its simulated twin are rounded the same way so they are comparable.

## 13.5 Testing and results

**Q26. How do you know the simulator is correct?**

Seven verification tests compare it with independently derived results: ceil(D/T) receives, P(D > T) redeliveries, recovery T − fD, heartbeat counts, and the Erlang C mean wait of an M/M/c queue (within 10%). Plus about 100 behaviour tests, and a real SQS run where attempt and extension counts matched.

**Q27. What is your strongest result?**

Under a workload shift, adaptive P90 cut duplicates from 20.6% to 5.9% versus a realistic pre-tuned configuration, in all 20 seeds. And the adaptive heartbeat with a learned budget beat the fixed heartbeat on duplicates, recovery and latency when workers can hang.

**Q28. Does adaptation always win?**

No. The oracle static policy has fewer duplicates; in stationary workloads adaptive P90 only matches pre-tuned after warm-up; and the fixed heartbeat recovers hung heavy-tailed jobs faster (16.5 s vs 20.3 s).

**Q29. Why 20 seeds and paired Wilcoxon tests?**

Pairing removes workload and failure luck. Wilcoxon does not assume normal differences. With 20 seeds the smallest exact p is about 2 × 10^-6, so significance is reachable after Holm correction; with 2 seeds it is not.

**Q30. What did the real AWS run show?**

Attempt and heartbeat-extension counts matched the simulator almost exactly and the duplicate ranking was preserved. Real SQS had fewer duplicates and longer heartbeat latencies than simulated. Recovery times are within noise because most policies had one crash.

**Q31. What does "18 of 18 hypotheses supported" really mean?**

Each declared paired test went in the expected direction with Holm-adjusted $p < 0.05$. H1–H4 are post hoc re-tests; H5–H18 were committed before their evaluation runs. Some effects are small (under 1 point).

## 13.6 Limitations and novelty

**Q32. What is novel here?**

Not the percentile idea itself (TCP timeouts and leases are older). The contribution is an interpretable per-class controller with a heartbeat budget learned from the tail and re-read dynamically, plus a verified, reproducible evaluation with strong baselines and a real-SQS check.

**Q33. What are the biggest limitations?**

Synthetic workloads, simulation-based evidence, one small real run, no literature comparison, and simplifications such as instant worker replacement and first-success deletion.

**Q34. How would you deploy this?**

Keep per-class histories in the worker fleet (or a shared store), compute the lease on receive, heartbeat with ChangeMessageVisibility, store committed job ids durably for idempotency, and configure a DLQ with a sensible maxReceiveCount.

## 13.7 Challenging questions

**Q35. Your oracle has fewer duplicates than your controller. Why not just use per-class static timeouts?**

The oracle has seen the evaluated jobs, which is impossible in practice. The realistic pre-tuned version matches adaptive P90 when nothing changes but fails badly after a shift (20.6% vs 5.9%). Adaptation buys robustness to change.

**Q36. If a short heartbeat lease is so good, why bother adapting the lease at all?**

The lease mostly matters for short classes, which finish inside it. What adaptation really adds is the per-class budget, which bounds hangs. The fixed-budget ablation shows that without hangs the budget does not matter, and with hangs it cuts hang recovery by about 18–21 s (README Phase 3).

**Q37. Your duplicate metric divides by attempts. Is that fair?**

It is the share of processing attempts that overlapped another; policies that create many duplicates also create many attempts, which slightly understates bad policies. The trade-off cost metric, in seconds per job, is a complementary view.

**Q38. Why might real SQS show fewer duplicates than the simulator?**

Real calls and polling add delay before a redelivered message is picked up; the harness starts the policy lease after receiving; SQS expiry timing is not exact. These were not measured separately, so they are hypotheses.

**Q39. Could the hypotheses be cherry-picked?**

Each was written into configs/full.json and committed before the run that tests it (commit hashes in methodology.md), tuning used separate seeds, and all 18 are reported with Holm correction. H1–H4 are honestly labelled post hoc.

**Q40. What happens when a class is rare?**

It stays in cold start longer (few samples), using the generous partial rule, so it rarely duplicates but recovers crashes more slowly. Listed as a known adaptive-policy failure mode.

**Q41. Why does Page–Hinkley use log durations?**

On a log scale a doubling is the same size for a 1 s and a 14 s class, so one threshold works for all classes; log-normal noise also becomes symmetric.

CHAPTER 14

# Quick Revision

## 14.1 One-page summary

| Question | Answer |
|---|---|
| Problem | One fixed SQS visibility timeout either duplicates long jobs (too short) or recovers crashes slowly (too long). |
| Idea | Learn the timeout per job class from recent durations (P90 + 2 s); better, use a short heartbeat lease with a per-class budget learned from P99. |
| How tested | Verified discrete-event simulator of SQS; 7,060 runs; 28 scenarios; 31 policies; 20 seeds; paired Wilcoxon + Holm; 18 declared hypotheses. |
| Main results | Shift: duplicates 5.9% (adaptive) vs 20.6% (pre-tuned). Hangs: adaptive heartbeat beats fixed heartbeat on duplicates, recovery and latency. Lowest trade-off cost in most scenarios. |
| Where it loses | Cold start; heavy-tailed jobs (P90 can't cover the tail); fixed heartbeat better for heavy-tailed hangs; oracle static has fewer duplicates. |
| Real check | One 40-job SQS run per policy: attempt and extension counts match; duplicate ranking preserved; fewer real duplicates; recovery too noisy to judge. |
| Limits | Synthetic workloads, simulation evidence, small real run, no literature review, simplifying assumptions. |

## 14.2 Complete working flow

1. Read the JSON config.
2. For each scenario and seed, generate the jobs (same for all policies).
3. Build the policies (fixed, oracle, pre-tuned, adaptive, heartbeat, variants).
4. For each policy, run the simulator: send events → pop earliest event → handle → dispatch to idle workers.
5. On receive: policy chooses timeout (and heartbeat interval and budget); failure model decides crash/hang; future events scheduled.
6. On expiry: if still current and not deleted, message becomes visible (duplicates or recovery).
7. On heartbeat: extend lease up to the budget.
8. On completion: commit once via the ledger, delete, teach the policy the duration.
9. Stop when all jobs are deleted or dead-lettered.
10. Summarise each run into metrics; aggregate across seeds; paired tests; Holm; primary hypotheses.
11. Write CSVs, plots and report.html.
12. Optional: replicate four policies on real SQS and compare with the simulated twin.

## 14.3 Concepts to remember

- Visibility timeout, receipt handle, at-least-once, idempotency, DLQ.
- Duplicate delivery vs duplicate processing vs duplicate committed result.
- Crash (lease end ends it) vs hang (budget ends it).
- Per-class rolling window; cold start; oracle vs pre-tuned baseline.
- Discrete-event simulation; common random numbers; paired tests; Holm; pre-declared hypotheses; Pareto front.

## 14.4 Key formulas and metrics

```
timeout   = clamp(P90(last 40 committed durations of class) + 2,  1, 120)
cold      = 1.5 x max(samples) + 2      (1-4 samples);   12 s with none
lease     = min(5, P90 + 2);   interval = 0.7 x lease;   budget = clamp(P99 x 1.25 + 2, lease, 40)
conformal = sorted[ceil((n+1) q) - 1];   cost-aware q* = 1 - p r / (1 - p),  p ~ Beta(1,19)
Page-Hinkley: U += x - mean - 0.05;  alarm if U - min U > 3   (x = log duration)
dup rate = overlapping attempts / attempts;   recovery = next receive - failure time
trade-off cost = (duplicate seconds + recovery seconds) / completed jobs
checks: receives = ceil(D/T);  recovery = T - fD;  Erlang C mean wait
```

## 14.5 Component-to-responsibility reference

| Component | Responsibility |
|---|---|
| workload.py | Seeded jobs; oracle and pre-tuned timeouts |
| failure_model.py | Crash/hang draws keyed by (seed, job, receive count) |
| policies/ | Timeout, heartbeat interval, budget; learning |
| simulator.py | Event-driven SQS queue + workers + ledger |
| metrics.py | Per-run metrics, aggregates, t intervals |
| stats.py | Wilcoxon, Holm, paired comparisons, Pareto, primary tests |
| experiment.py | Orchestration, plots, HTML report, CLI |
| replication.py | Real/fake SQS run with threads, twin simulation, comparison |

## 14.6 Most important files and functions

- `SQSSimulator._dispatch`, `_on_visibility_expire`, `_apply_extension`, `_on_complete` (simulator.py)
- `AdaptivePercentilePolicy.decide`, `_cold_start_decision`, `_page_hinkley` (adaptive_percentile.py)
- `AdaptiveHeartbeatPolicy.max_total_visibility_for` (adaptive_heartbeat.py)
- `FailureModel.sample_for` (failure_model.py); `pretuned_class_timeouts` (workload.py)
- `summarize_run` (metrics.py); `primary_tests`, `wilcoxon_signed_rank`, `holm_adjust` (stats.py)
- `run_experiments`, `build_policies` (experiment.py); `ReplicationRun._process` (replication.py)

## 14.7 Novelty in two sentences

An interpretable, per-class visibility-timeout controller that learns from recent durations, and a heartbeat variant whose per-class budget is learned from the tail and re-read at every heartbeat, so crashes are caught within a short lease and hung workers are reclaimed after about their class's worst normal duration. Its value is shown by a verified, reproducible simulation study with strong baselines and pre-declared hypotheses, plus a small real-SQS replication.

## 14.8 The two-minute explanation

*"When a worker takes a message from Amazon SQS, the message is hidden for a visibility timeout. If the timeout is too short, the message reappears while the worker is still busy and another worker repeats the job. If it is too long, a job whose worker crashed sits hidden for a long time before anyone retries it. Most systems use one fixed timeout for every job, which can't suit both one-second and fifteen-second jobs.*

*My project builds a controller that sets the timeout per job type from the 90th percentile of recent durations plus a margin, and a stronger version that keeps a short five-second lease, renews it with heartbeats, and stops renewing after a budget learned from the 99th percentile, so hung workers are reclaimed quickly.*

*To test it fairly I wrote a discrete-event simulator of SQS, verified against queueing theory, and ran over seven thousand simulations across 28 scenarios and 20 seeds, comparing against fixed timeouts and both a realistic and an oracle per-class configuration. I declared 18 hypotheses in advance and all were supported after Holm correction. For example, after a workload shift the adaptive controller had 5.9% duplicate processing against 20.6% for a pre-tuned configuration, and the adaptive heartbeat beat a fixed heartbeat on duplicates, recovery and latency when workers can hang.*

*It does not always win: the fixed heartbeat is still better for heavy-tailed hangs, and an oracle static policy has fewer duplicates. Finally, I replayed four policies on a real SQS queue: the attempt and heartbeat counts matched the simulator, though real SQS produced fewer duplicates, and one small run is not enough to judge recovery times."*

# Glossary

| Term | Meaning |
|---|---|
| **Adaptation time** | After a shift, time until 5 consecutive predictions are within 20% of the new P90. |
| **Aggregate** | Summary across seeds: mean, SD and intervals. |
| **API call** | One request to SQS (send, receive, delete, change visibility). |
| **At-least-once delivery** | Every message is delivered one or more times. |
| **Bootstrap interval** | Confidence interval from resampling the seeds many times. |
| **Budget (visibility budget)** | Maximum total hidden time from processing start; caps heartbeat extensions. |
| **Class (job class)** | The job_type: short, medium, long or heavy. |
| **Clamp** | Limit a value to a range, here 1–120 s. |
| **Cold start** | Period with fewer than min_history samples for a class. |
| **Common random numbers** | Same random draws for every policy in the same situation. |
| **Committed result** | The single accepted result of a job, recorded in the ledger. |
| **Conformal quantile** | The ceil((n+1)q)-th smallest sample; covers the next value with probability ≥ q if data are exchangeable. |
| **Cost-aware quantile** | $q^* = 1 - p \cdot r/(1 - p)$, chosen from the failure rate p and cost ratio r. |
| **Coverage** | Share of first attempts finishing within the predicted quantile. |
| **Crash** | Worker dies part-way; heartbeats stop. |
| **Dead-letter queue (DLQ)** | Queue for messages received more than maxReceiveCount times. |
| **Discrete-event simulation** | Simulation that jumps from event to event in time order. |
| **Duplicate processing** | An attempt that starts while another attempt of the same job is running. |
| **Dynamic budget** | Budget recomputed at every heartbeat. |
| **Erlang C** | Formula for the mean waiting time in an M/M/c queue; used to verify the simulator. |
| **Exchangeable** | Order does not matter statistically; true of stationary independent data. |
| **Hang** | Worker stops progressing but stays alive and keeps heartbeating. |
| **Heartbeat** | Periodic ChangeMessageVisibility call that extends the lease. |
| **Holm correction** | Step-down adjustment of p-values controlling family-wise error. |
| **Idempotency** | Applying an operation once or many times has the same effect. |
| **Lease** | One hidden period; initial timeout or extension length. |
| **Ledger** | In-memory set of committed job ids; first writer wins. |
| **Liveness probe** | Supervisor that kills a hung worker after a set time. |
| **Makespan** | Time until all jobs are done. |
| **maxReceiveCount** | Receives allowed before a message is dead-lettered. |
| **Oracle baseline** | Static per-class timeouts fitted to the evaluated jobs themselves (unrealistic upper bound). |
| **Page–Hinkley test** | Sequential change-detection test on cumulative deviations from the running mean. |
| **Paired test** | Test on per-seed differences between two policies. |
| **Pareto optimal** | Not beaten on both metrics by any other policy. |
| **Percentile (P90, P99)** | Value below which 90% (99%) of observations fall. |
| **Poison message** | Message that fails on every attempt. |

| Term | Meaning |
|---|---|
| **Pre-tuned baseline** | Static per-class timeouts fitted to the first 20% of an independent pilot workload. |
| **Primary hypothesis** | One of the 18 declared tests, corrected as its own family. |
| **Receipt handle** | Token identifying one delivery; needed to delete or extend. |
| **Recovery time** | Time from a worker failure to the next receive of that job. |
| **Rolling window** | Most recent W durations per class (deque with maxlen). |
| **Seed** | Number that fixes a random sequence. |
| **Standard queue** | SQS queue type with at-least-once, best-effort-ordered delivery. |
| **Steady state** | Metrics after excluding the first 20% of jobs (warm-up). |
| **Trade-off cost** | Duplicate-processing seconds plus recovery seconds, per completed job. |
| **Twin (simulated twin)** | Simulator run with the same jobs and failure draws as a real replication run. |
| **Visibility timeout** | Time a received message stays hidden before it can be received again. |
| **Wilcoxon signed-rank test** | Non-parametric paired test based on ranks of differences. |

# Appendix A — Command Reference

| Goal | Command |
|---|---|
| Install dependencies | `python -m pip install -r requirements.txt` |
| Run all tests | `python -m pytest -q` |
| Quick demonstration | `python -m src.experiment --config configs/smoke.json --output results/smoke --plots` |
| Full experiment | `python -m src.experiment --config configs/full.json --output results/full --plots` |
| Print full aggregate table | add `--verbose` |
| Offline replication dry run | `python -m src.replication --fake --time-scale 0.2` |
| Real replication | `python -m src.replication --queue-url <URL> --region <REGION> --no-open` |
| Rebuild a replication report | `python -m src.replication_report results/aws_replication` |
| CloudShell: create queue | `aws sqs create-queue --queue-name adaptive-visibility-test` |
| CloudShell: delete queue | `aws sqs delete-queue --queue-url "$QURL"` |

# Appendix B — Source File Index

| File | Key definitions (line) |
|---|---|
| src/message.py | Message (8), DeliveryAttempt (20) |
| src/workload.py | WorkloadSpec (11), _duration (21), generate_workload (36), oracle_class_timeouts (93), pretuned_class_timeouts (104) |
| src/failure_model.py | FailureDecision (10), FailureModel (16), from_level (35), sample_for (41), sample (48) |
| src/idempotency.py | InMemoryIdempotencyLedger (6), commit_once (13) |
| src/policies/base.py | TimeoutDecision (8), VisibilityPolicy (14) |
| src/policies/fixed.py | FixedPolicy (6), MeanFixedPolicy (17) |
| src/policies/class_based.py | ClassBasedPolicy (6) |
| src/policies/heartbeat.py | FixedHeartbeatPolicy (6) |
| src/policies/adaptive_percentile.py | AdaptivePercentilePolicy (11), decide (123), _cold_start_decision (135), target_quantile (143), failure_rate (152), _quantile (156), margin (164), observe (171), _page_hinkley (179) |
| src/policies/adaptive_heartbeat.py | AdaptiveHeartbeatPolicy (9), decide (94), heartbeat_interval_for (98), max_total_visibility_for (101), budget_quantile (117) |
| src/simulator.py | SQSSimulator (19), _dispatch (108), _attempt_duration (174), _record (183), _on_failure (214), _on_hang (226), _release_hung (238), _on_complete (252), _delete (269), _on_outage (277), _on_visibility_expire (288), _on_heartbeat (299), _apply_extension (308), run (336) |
| src/metrics.py | summarize_run (10), _dead_letter_metrics (94), _steady_state_metrics (125), t_critical (197), aggregate_runs (217) |
| src/stats.py | wilcoxon_signed_rank (49), holm_adjust (85), paired_comparisons (98), pareto_optimal (152), primary_tests (173) |
| src/experiment.py | DEFAULT_SCENARIOS (26), DEFAULT_ADAPTIVE_VARIANTS (41), build_policies (62), realism_options (117), run_experiments (138), generate_plots (234), write_html_report (351), main (440) |
| src/replication.py | FakeSQSClient (49), IntegerTimeouts (115), replication_policies (152), ReplicationConfig (167), ReplicationRun (179), _process (252), compare (352), run_replication (367), main (391) |
| src/replication_report.py | _plots (28), write_replication_report (103) |
| src/aws_sqs_adapter.py | AWSValidationConfig (11), Boto3SQSAdapter (18) |
| docs/ | architecture.md, methodology.md, limitations.md, aws_replication.md, iam-policy.json |
| configs/ | smoke.json (50 jobs, 6 workers, 2 seeds, 4 scenarios), full.json (200 jobs, 10 workers, 20 seeds, 28 scenarios, H1–H18) |